

中国研究生操作系统开源创新大赛

项目说明书

EulerPilot:

面向 openEuler 的自适应资源管控 Agent

作品名称: EulerPilot: 面向 openEuler 的
自适应资源管控 Agent

参赛队伍:

我为大红而来

完成日期:

2026年7月

目 录

第 1 章 绪 论.....	1
1.1 目的与意义.....	1
1.2 项目背景.....	1
1.2.1 普遍问题.....	1
1.2.2 同类机制与差异.....	2
1.2.3 需求与约束.....	3
1.3 适用范围与边界.....	3
第 2 章 技 术 理 论.....	5
2.1 设计思想.....	5
2.1.1 观测模型与增量计算.....	5
2.1.2 Workload 分类与控制资格.....	6
2.1.3 压力判定与滞回.....	6
2.1.4 PsiGate 四状态机.....	7
2.1.5 双执行后端.....	7
2.1.6 跨 Skill 事务与回滚.....	8
2.2 技术路线.....	9
2.2.1 eBPF 观测与内核数据采集.....	9
2.2.2 Agent Framework 与能力感知启动.....	10
2.2.3 Workload 分析与 PsiGate 自适应决策.....	10
2.2.4 TargetResolver 与统一目标解析.....	11
2.2.5 多资源执行与双后端控制.....	11
2.2.6 Policy Engine、审计与事务回滚.....	12
2.2.7 测试门禁与证据交付.....	12

2.3 技术介绍	13
2.3.1 openEuler 与 Linux 内核基础	13
2.3.2 eBPF、BTF、CO-RE 与 libbpf	13
2.3.3 PSI 与 cgroup v2 资源控制	15
2.3.4 sched_ext 可编程调度	15
2.3.5 Web Console 与证据交付技术	16
2.4 代码原创说明	16
第 3 章 项目介绍	18
3.1 功能介绍	18
3.2 Agent Framework	22
3.2.1 统一 Skill 生命周期	22
3.2.2 能力探测与安全启动	23
3.2.3 任务采样、身份与分类	24
3.2.4 PSI Gate 与自适应阈值	26
3.3 Resource Control Skill	28
3.3.1 Profile 与 CPU 控制	28
3.3.2 Memory 与 IO 控制	29
3.3.3 路径边界、回读和回滚	30
3.4 TargetResolver	31
3.4.1 cgroup、PID 与容器	32
3.4.2 Kubernetes Pod 与 host veth	32
3.5 sched_ext 调度增强	33
3.5.1 能力条件和 Map 联动	34
3.5.2 DSQ 策略和失效保护	34
3.6 Network Policy Skill	35

3.6.1 connect4、TC QoS 与 XDP	36
3.7 Security Policy Skill.....	38
3.7.1 LSM、规则与异常聚合	38
3.7.2 加载、消费与清理	39
3.8 Policy Engine 与证据链	40
3.8.1 跨 Skill 事务	40
3.8.2 AuditBus、ActionJournal 与门禁	42
3.9 Web Console	42
3.9.1 技术栈、定位与页面结构	43
3.9.2 API、白名单任务与安全控制	48
3.10 模块协作接口与运行状态	49
3.10.1 Runtime 主循环的数据组织	50
3.10.2 Resource 操作契约	51
3.10.3 故障可见性与降级矩阵	52
3.10.4 配置校验与验收入口	53
3.11 端到端功能闭环	54
第 4 章 实验测试.....	56
4.1 测试说明	56
4.1.1 测试分层与可复现策略	56
4.1.2 验证环境	57
4.1.3 有效性门禁	58
4.1.4 Web Console 前后端测试与环境连通性核验	58
4.2 测试结果	59
4.2.1 最终质量门禁	59
4.2.2 Redis 混部后端对照	60

4.2.3 人工静态与 Agent 动态	61
4.2.4 压力梯度与闭环恢复	64
4.2.5 Nginx 与 throughput-first 边界	65
4.2.6 Agent 自身开销	65
4.2.7 资源、网络、安全和真实 Pod 验证	66
4.2.8 结果限制	66
第 5 章 实现难点说明	68
5.1 跨内核能力差异与优雅降级	68
5.2 任务身份与 PID 复用	68
5.3 TargetResolver 的容器与 Pod 映射	68
5.4 多资源事务一致性	68
5.5 eBPF Hook 生命周期与无残留	69
5.6 实验有效性与负结果报告	69
5.7 控制台的最小权限交互	69
第 6 章 总 结	70
6.1 完成度	70
6.2 主要结论	70
6.3 创新点归纳	70
6.4 后续工作	71
6.5 致谢与开源边界	72
参 考	73

第 1 章 绪 论

1.1 目的与意义

openEuler 是面向服务器、云计算、边缘和云原生场景的开源操作系统。它的优势在于内核能力更新较快，软件包和社区文档比较完整，能够直接提供 cgroup v2、PSI、eBPF、BTF、TC/XDP 等系统接口，也为 sched_ext、BPF LSM 等新机制提供验证空间。对于资源管控类项目来说，openEuler 既有稳定的发行环境，也能支撑较深入的内核机制实验。

不过，操作系统提供的是底层机制，真正落到业务现场时，还需要有人持续观察系统压力，判断是哪类任务在抢资源，决定该调 CPU、内存、IO 还是网络策略，并在调整后确认业务是否恢复。服务器混部和容器化部署越普遍，这个过程越难靠人工完成：在线服务、批处理、日志采集、CI 构建和临时实验进程可能同时运行，压力变化快，目标还可能藏在容器或 Kubernetes Pod 里，问题定位和恢复都需要更清楚的自动化链路。

EulerPilot 的目标是构建一个面向 openEuler 的资源管控 Agent。它把内核观测、用户态判断、受控执行、结果校验和回滚恢复连成一个闭环：先通过 eBPF、PSI 和 cgroup 状态识别系统压力，再由 Agent 判断关键 workload 是否受到影响，最后通过统一 Skill Framework 执行 CPU、内存、IO、网络和安全相关动作。对使用者来说，它解决的是比较直观的问题：业务开始卡顿时，系统能够更快看出压力来源，自动选择有限、可审计、可恢复的控制动作，并把每一步留下证据。

1.2 项目背景

1.2.1 普遍问题

一台正常服务器很少只运行一个程序。在线接口、数据库、日志采集、备份、CI 构建、容器任务和临时实验进程经常同时存在。后台任务一旦抢占 CPU 时间、内存带宽或 IO 队列，前台服务就可能表现为请求变慢、P99 抬高、偶发超时，用户看到的就是“机器没死，但业务开始卡”。这类问题具有普遍性，并不只属于 Redis 或 Nginx；本文使用 Redis、Nginx 与后台压力作为实验载体，是为了把资源争抢、延迟抬高和恢复效果量化出来，项目本身面向更一般的混部资源治理。

1.2.2 同类机制与差异

EulerPilot 与传统静态调优、单一 eBPF 工具和独立调度器的差异主要体现在跨模块闭环、统一目标解析以及可验证回滚三个方面，如表 1 所示。

表 1 相关技术路线比较

方向	优势	局限	EulerPilot 的取舍
静态 cgroup 配额	内核原生、接口稳定、易回读	参数需要人工设定，可能长期过度限制	保留为主执行后端，并由 Agent 按 profile 写入与恢复
用户态调度框架	策略迭代快，可引入应用语义	部署与内核接口复杂，错误隔离要求高	借鉴用户态策略思想，但把写入限定在可探测能力与回滚边界内
sched_ext/scx	BPF 定义调度策略，可动态装卸并在错误时回退	依赖 CONFIG_SCHED_CLASS_EXT 等内核配置；策略质量与 workload 相关	仅在能力探测通过时启用，SP4 使用官方源码自编译启用内核复核
单一 eBPF 工具	观测或过滤路径短，开发目标明确	通常缺少跨模块事务、目标解析与交付证据	通过 SkillManager、Policy Engine、AuditBus 和 ActionJournal 形成统一生命周期

ghOSt 将调度策略委托给用户态，证明了通用内核与快速策略迭代可以结合^[1]；Linux sched_ext 提供可动态装卸的 BPF 调度类，并在错误、任务停顿或主动退出时恢复到公平调度类。EulerPilot 的工作重点放在 openEuler 环境适配、压力门控、双后端、跨 Skill 联动、真实 Pod 目标解析、回滚与证据门禁这些工程闭环上。

1.2.3 需求与约束

项目需求均能对应到明确的源码模块、配置入口和验证证据，形成从需求到实现再到验收的可追溯关系，如表 2 所示。

表 2 需求到实现的可追溯映射

需求	设计响应	源码或证据入口
低开销观测	sched_wakeup/sched_switch/sched_migrate_task + PSI；用户态聚合增量	bpf/workload_observer.bpf.c； agent/src/runtime.cpp
自适应控制	workload 分类、阈值、滞回、PsiGate 四状态机	agent/src/runtime.cpp； agent/src/psi_gate.cpp
多资源执行	CPU/Memory/IO、TC/XDP、BPF LSM、sched_ext	agent/src/executors.cpp； agent/src/builtin_skills/
安全作用域	显式 target_ref、能力探测、默认 dry-run/disabled	agent/src/target_resolver.cpp； configs/agent.yaml
失败恢复	旧值/句柄日志、逆序 rollback、cleanup 脚本	agent/src/action_journal.cpp； SkillManager::rollback_all
可复现验证	manifest 白名单、正式多轮实验、无残留门禁	configs/final_evidence_manifest.json ； scripts/final_quality_gate.sh

1.3 适用范围与边界

- 适用：openEuler 24.03 LTS SP3/SP4 上的服务器、实验节点和隔离的 Kubernetes lab 环境。
- 稳定主路径：cgroup v2 的 cpu.weight、cpu.max、memory.high/low/max 和 io.weight/io.max；cpuset.cpus 保留显式实验接口，封版默认关闭。
- 增强路径：sched_ext/scx 依赖启用 CONFIG_SCHED_CLASS_EXT、BPF、BTF 等配置的内核；项目不声称 SP4 发行默认内核直接支持。

- 网络与安全能力默认关闭；只有显式配置 `target`、规则与 `active/enforce` 模式才改变系统状态。

第 2 章 技术理论

2.1 设计思想

EulerPilot 采用能力感知、证据驱动和保守执行三项原则。能力感知意味着每个 Skill 在 init/start 前先 probe BTF、cgroup v2、PSI、BPF LSM、TC、XDP、sched_ext 或 Kubernetes 条件；证据驱动意味着压力模式至少由 workload 存在性与 PSI、等待时间、后台运行时间等信号组合判定；保守执行意味着未知 workload 不进入自动控制集合，配置默认 dry-run，网络和安全策略默认 disabled。

系统以观测数据为起点，依次完成任务分析、门控决策、目标解析和受控执行，并通过审计与回滚构成完整闭环，如图 1 所示。

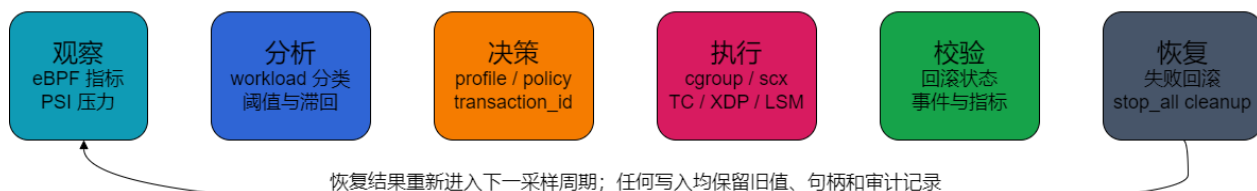


图 1 自适应控制与可回滚事务闭环

2.1.1 观测模型与增量计算

内核侧 observer 在调度事件上更新任务统计，用户态每个采样周期读取 BPF map，并对累计计数做安全差分。主要指标包括 wakeup_count、total_wait_ns、runtime_ns、ctx_switch_count 与 migrate_count。为避免 PID 复用造成的错误增量，代码优先使用 BPF 提供的 start_boottime_ns 标识任务代际；缺失时使用用户态 generation cookie，并在累计值回退或身份变化时重置基线。相关边界由 tests/unit/test_runtime_policy.cpp 覆盖。

PSI 读取 /proc/pressure/cpu、memory、io 的 some/full 与 avg10/avg60/avg300/total。Linux 内核文档说明，PSI 用停顿时间描述资源争用造成的生产力损失，并支持阈值触发与 cgroup 级接口^[2]。EulerPilot 当前用 CPU some.avg10 与显式阈值判断 cpu_psi_high，同时保留 poll trigger 作为 PsiGate 的另一项证据。

2.1.2 Workload 分类与控制资格

不同 WorkloadClass 具有不同的识别依据和控制边界，其中 Unknown 类保持观察，避免在证据不足时扩大干预，如表 3 所示。

表 3 Workload 分类与控制边界

类别	识别线索	默认控制资格	典型 profile
LATENCY_SENSITIVE	redis-server、nginx 等受管服务名 或明确特征	受管目标可进入 Gate； 未知相似进程仅诊断	normal_profile / latency_profile
BACKGROUND_NOISY	stress-ng、构建辅助进程及干扰 特征	只有受管目标参与 Gate 与写入	latency_profile / mixed_profile
THROUGHPUT_BATCH	sysbench、make 等长运行低唤醒 任务	默认不触发延迟门控； 显式 throughput-first 才 使用增强 profile	normal_profile / throughput_profile
UNKNOWN	未命中受管名称和安全特征	不自动写入	observe-only

分类结果同时包含 managed_target、gate_relevant 和 gate_reason。这样做是为了把“看起来像某类负载”与“允许改变它的资源配置”分开：前者由行为评分和进程语义共同决定，后者还要经过目标白名单、作用域和安全模式检查。详细评分公式和触发标准在第 3 章“任务采样、身份与分类”中展开。环境变量

EULERPILOT_FEATURE_CLASSIFY_UNKNOWN 可启用未知任务的特征诊断，但 runtime.cpp 明确把这些结果标记为 diagnostic-only，不因此取得写权限。

2.1.3 压力判定与滞回

默认阈值在 eulerpilot.hpp 中为 CPU PSI 0.10、延迟任务等待增量 5,000,000 ns、后台运行增量 4,000,000 ns。若启用 EULERPILOT_ADAPTIVE_THRESHOLDS 且设置 warmup_cycles，系统在 warmup 期间保持 dry-run，并以样本 P95 校准等待、后台运行与 PSI 阈值；CPU PSI 校准值限制在 0.001 至 0.50。显式环境变量阈值优先，不被自动校准覆盖。

控制模式 `derive_desired_mode` 先检查 `latency_exists` 与 `background_exists`。两类 `workload` 同时存在且“延迟等待高 + (CPU PSI 高或后台运行高)”时进入 MIXED；仅有部分压力证据时进入 LATENCY；没有足够证据时回 NORMAL。
`update_mode_with_hysteresis` 对状态转换使用连续窗口计数，避免单周期毛刺导致反复写入。

2.1.4 PsiGate 四状态机

PsiGate 通过 NORMAL、ARMED、ACTIVE 和 COOLDOWN 四态以及连续窗口条件抑制瞬时压力抖动，如表 4 所示。

表 4 PsiGate 当前源码默认状态转换

状态	进入条件	输出	退出条件
NORMAL	没有满足激活组合	<code>sched_ext_normal</code>	出现候选压力证据进入 ARMED
ARMED	背景任务存在，压力证据开始累计	<code>sched_ext_armed</code>	默认连续 2 个窗口满足后进入 ACTIVE；证据消失回 NORMAL
ACTIVE	背景存在且至少两类证据满足	<code>sched_ext_active</code>	默认连续 3 个恢复窗口后进入 COOLDOWN
COOLDOWN	压力刚消退	仍保持 <code>active</code> profile，防止抖动	默认 2000 ms 后回 NORMAL；压力复现可回 ACTIVE

2.1.5 双执行后端

CgroupExecutor 通过 `cgroup v2` 控制文件执行资源 profile。内核文档将 `cgroup v2` 定义为统一层级上的进程组织与资源分配机制，并规定 `cpu.max` 等接口语义。EulerPilot 写入前检查文件白名单、读取旧值，写入后回读验证；`cpu.max` 的“max”兼容“max period”显示形式，`io.max` 按设备行核验。

CPU、Memory、IO 和 sched_ext 四类执行面均保存与自身对象相匹配的旧值或句柄，为精确恢复提供依据，如表 5 所示。

表 5 资源执行面与回滚信息

资源	控制文件	用途	回滚证据
CPU	cpu.weight、cpu.max、 cpuset.cpus 显式实验，默认关闭	相对权重、带宽上限、 CPU 集合	旧值 + 目标 cgroup + 回 读值
Memory	memory.high、memory.low、 memory.max	软限制、保护值、硬上 限	memory.high guard + old/new values
IO	io.weight、io.max	权重和设备级带宽/IOPS	设备行匹配 + 恢复值
sched_ext	class_map、gate_state_map、 stats	任务分类、门控状态、 调度计数	pinned map 快照 + scheduler 生命周期

ScxExecutor 负责启动或连接 scx_eulerpilot，将用户态分类写入 class_map，并同步 gate_state_map。Linux sched_ext 文档说明，该调度类由 BPF 程序定义，可动态装卸，在检测到内部错误或可运行任务停顿时回退到公平调度类。因此本项目把 scx 定位为受能力检测约束的增强后端，发行环境是否可用由内核配置和构建产物共同决定。

2.1.6 跨 Skill 事务与回滚

Policy Engine 从 security_policy 的 JSONL 事件中读取触发条件，生成 transaction_id 与 trigger_event_id，然后按 YAML 动作序列调用 resource_control 或 network_qos。每个动作只允许访问明确白名单文件；memory.high 写入还会检查 memory.max。动作成功后记录 old_values、new_values 和 handles；任一后续动作失败时，restore_actions 逆序恢复同一事务已完成动作。

```
security_policy anomaly event
-> policy_engine match policy_id
-> create transaction_id
-> resource_control: write cgroup value and verify
-> network_qos: attach/configure on resolved host veth
-> append AuditBus + ActionJournal records
-> on failure/stop: restore in reverse order
```

2.2 技术路线

EulerPilot 的技术路线围绕“观测、判断、执行、验证、恢复”展开，技术架构如图 2 所示。系统首先通过 eBPF 调度事件、PSI 和 cgroup 状态获取工作负载运行证据，再由 C++17 Agent 完成任务身份维护、增量计算、Workload 分类和压力门控；随后，SkillManager 根据能力探测、目标作用域和策略配置选择 cgroup v2 稳定路径或 sched_ext 增强路径，并按需联动 TC/XDP 与 BPF LSM。所有执行动作均经过白名单校验、写后回读、审计记录与逆序回滚，测试结果最终汇入 Evidence 和质量门禁。

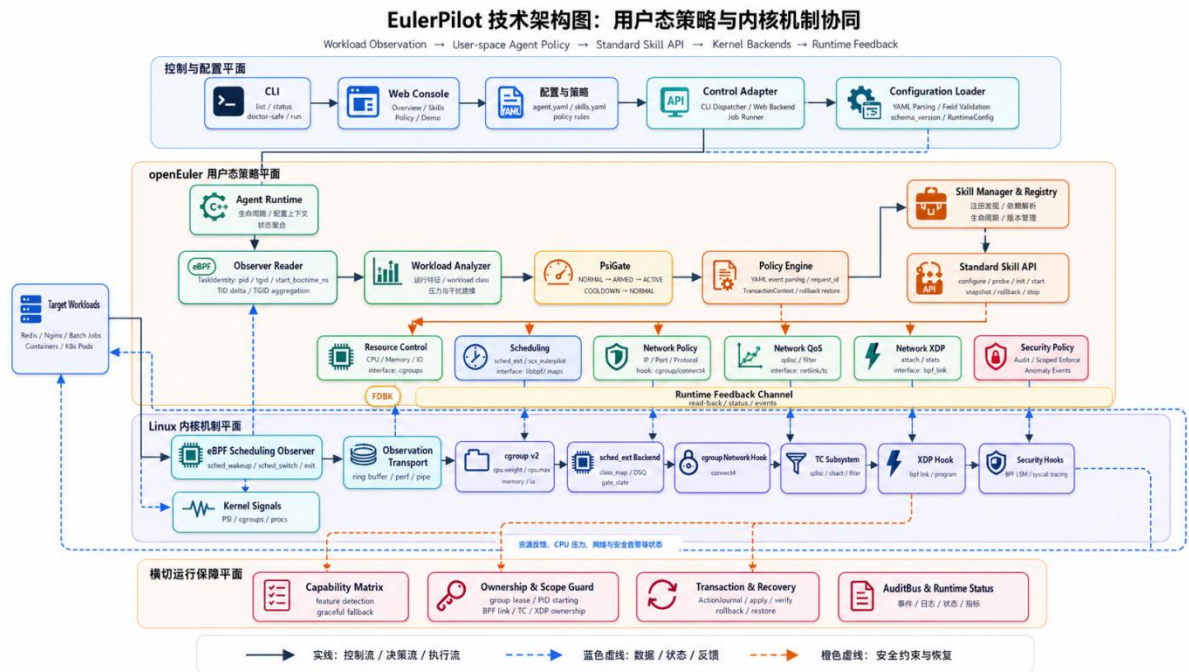


图 2 技术架构图

图 2 所示技术架构可划分为内核观测、Agent 编排、自适应决策、目标解析、多资源执行以及证据交付等相互衔接的环节。各环节共享统一的任务身份、目标引用和事务标识，内核观测结果必须经用户态策略和安全门禁后才能转换为系统状态变更，防止观测程序直接触发无边界控制。具体技术路线如下。

2.2.1 eBPF 观测与内核数据采集

项目使用 clang 编译 bpf/*.bpf.c 与 sched/scx_eulerpilot.bpf.c，利用目标内核 BTf 生成 vmlinux.h，并通过 bpftool 生成 skeleton。用户态使用 libbpf 完成 BPF 对象加载、程序 attach、BPF map 访问、ring buffer 消费以及 link 清理，使内核观测程序与 Agent 之间形成

稳定的数据通道。基于 BTF 和 CO-RE 的类型适配降低了不同 openEuler 内核版本之间的数据结构差异，但在程序加载前仍需检查 BTF、目标 Hook 和相关内核配置是否可用。

内核侧 workload_observer 挂接 sched_wakeup、sched_switch 和 sched_migrate_task 等调度事件，按任务累计唤醒次数、等待时间、运行时间、上下文切换次数和迁移次数；用户态 Runtime 按采样周期读取累计值并计算本周期增量。资源压力同时由 /proc/pressure/cpu、memory 和 io 提供，cgroup 状态用于补充目标归属和控制器能力。该设计将高频事件的轻量累计保留在内核侧，将身份校验、分类、阈值和策略计算放在用户态，在控制观测开销的同时保留策略迭代空间。

2.2.2 Agent Framework 与能力感知启动

用户态控制核心采用 C++17 实现，并从 agent.yaml 与 skills.yaml 读取运行参数、Skill 配置和依赖关系。CapabilityDetector 逐项探测 BTF、ring buffer、cgroup v2、PSI、BPF LSM、TC、XDP、sched_ext 以及 Kubernetes 工具链；SkillRegistry 维护 Skill 类型与工厂函数之间的映射，SkillManager 根据 dependencies() 完成依赖拓扑校验和启动排序。

所有 Skill 遵循 configure、probe、init、start、snapshot、rollback 和 stop 的统一生命周期。configure 仅解析和校验配置，probe 确认环境能力与目标条件，init 和 start 负责准备对象并进入运行状态，snapshot 输出结构化状态，rollback 与 stop 负责恢复副作用和释放句柄。当能力缺失或依赖不满足时，模块返回 unavailable、blocked 或降级原因，并停止进入执行阶段；默认 dry-run 和安全诊断入口保证系统在未知环境中不会直接修改内核状态。

2.2.3 Workload 分析与 PsiGate 自适应决策

Runtime 以 start_boottime_ns 作为任务代际标识；当该字段不可用时，使用用户态 generation cookie 补充身份判断。若累计计数回退、任务代际变化或 PID 被复用，系统会重置该任务的采样基线，避免把不同进程的数据错误合并。Workload Analyzer 根据等待、运行、唤醒、上下文切换和迁移等增量形成 LatencySensitive、ThroughputBatch、BackgroundNoisy、MixedService 或 Unknown 分类，并同步给出 managed_target、gate_relevant 和 gate_reason。分类结果仅说明任务特征，只有显式受管且通过控制资格检查的目标才可进入主动执行。

PsiGate 使用 NORMAL、ARMED、ACTIVE 和 COOLDOWN 四状态抑制瞬时压力抖动，并组合 workload 存在性、延迟等待、后台运行增量、CPU PSI 及 poll trigger 等证据决定是否激活控制。系统既支持固定阈值，也可在 warmup_cycles 期间保持 dry-run，并以预热样本 P95 校准未被显式环境变量覆盖的阈值。最终决策被转换为 normal、latency、mixed 或 throughput 等 profile，使“识别任务类型”“判断是否需要干预”和“选择执行参数”保持相互分离。

2.2.4 TargetResolver 与统一目标解析

TargetResolver 负责把策略中的显式 target_ref 转换为可执行的内核对象。资源控制场景可接收 cgroup 路径、PID、container 或 container_id，并解析为对应的 cgroup v2 路径；Kubernetes 场景可根据 Pod 信息定位容器 cgroup，网络策略则继续解析 Pod 或容器对应的 host veth 与 ifindex。通过统一解析层，资源、网络和安全 Skill 无需分别实现一套目标识别逻辑。

目标解析结果必须经过存在性、路径边界、接口状态和作用域检查，未知进程、无法确认归属的容器以及未显式配置的 Pod 不会获得默认写权限。同一目标标识会被写入运行状态、AuditBus、ActionJournal 和回滚记录，使一次动作的策略来源、实际对象、执行结果和恢复对象能够相互对应。

2.2.5 多资源执行与双后端控制

Resource Control 将 profile 转换为 CPU、Memory 和 IO 控制动作。发行内核上的稳定主路径基于 cgroup v2：CPU 使用 cpu.weight 与 cpu.max 调整相对份额和带宽上限，Memory 使用 memory.high、memory.low 和 memory.max 表达软限制、保护值与硬上限，IO 使用 io.weight 和 io.max 实施权重或设备级带宽/IOPS 控制；cpuset.cpus 保留显式实验接口，但封版默认关闭。CgroupExecutor 在写入前检查控制文件白名单并保存旧值，写入后通过控制文件回读核验结果，其中 memory.high 需要与 memory.max 进行边界检查，io.max 按目标设备行验证。

sched_ext/scx 作为满足内核能力条件时启用的增强的调度路径。ScxExecutor 负责启动或连接 scx_eulerpilot，将 Workload 分类和 Gate 状态写入 class_map 与 gate_state_map，并通过 DSQ 策略区分不同任务类别；当调度器出现内部错误、任务停顿或主动退出时，系统

依赖 sched_ext 的失效保护回到公平调度路径。Network Skill 可通过 connect4、TC QoS 和 XDP 实施连接控制与流量治理，Security Skill 通过 BPF LSM 执行规则与异常采集；这些能力均默认关闭，只有在显式目标、规则及 active/enforce 条件满足时才进入实际执行。

2.2.6 Policy Engine、审计与事务回滚

Policy Engine 从 security_policy 产生的 JSONL 事件中读取触发条件，匹配 YAML 策略后生成 transaction_id 与 trigger_event_id，并按照动作序列调用 resource_control、network_qos 等 Skill。每个动作只允许访问明确白名单中的控制文件、接口或规则对象，执行成功后记录 old_values、new_values 和 handles；任一后续动作失败时，系统对同一事务中已经完成的动作执行逆序恢复，避免资源和网络状态只完成部分更新。

AuditBus 负责记录结构化事件与结果，ActionJournal 保存具体动作、目标、旧值、新值、句柄和恢复状态，控制文件回读、BPF map 状态与 Runtime Status 共同构成执行校验。Agent 停止、实验结束或异常退出时，SkillManager 依次执行 rollback_all 与 stop_all，并依据对象所有权清理 link、qdisc、map 和临时 cgroup。这样，正常执行、失败处理和停止清理均落在同一事务与证据边界内。

2.2.7 测试门禁与证据交付

测试脚本为每次运行生成 run_manifest、CSV、JSONL、日志和 report.md，并通过统一运行标识关联配置、环境、动作与结果。collect_final_evidence.py 依据 final_evidence_manifest.json 对白名单内的必需证据进行检查，final_quality_gate.sh 同时核验构建、smoke 测试、默认安全配置与无残留状态。SP4 主验证环境用于确认完整功能链，SP3 兼容环境用于验证稳定主路径和降级行为，路径可用性、性能收益和环境限制分别记录。

测试门禁和证据交付只接收已经生成的运行材料，包括结构化状态、JSONL 事件、CSV、run_manifest、report.md 和 release manifest。证据工具按照白名单清单读取数据，缺失、不可用、无效和退化结果都保留独立状态，避免在整理报告时把历史结果或未认可结果混入最终结论。

2.3 技术介绍

EulerPilot 建立在 openEuler 与 Linux 内核提供的可观测、资源控制、可编程调度、网络治理和安全扩展能力之上。与 2.2 节侧重说明项目各模块如何形成端到端技术路线不同，本节从基础技术的功能定位、工作方式及其在 EulerPilot 中的作用出发，对项目使用的主要技术进行介绍。项目技术体系由内核侧机制、用户态 Agent、配置与策略、执行与回滚以及证据交付共同组成。

2.3.1 openEuler 与 Linux 内核基础

openEuler 是 EulerPilot 的主要运行与验证平台，项目直接使用 Linux 内核提供的调度事件、BPF、BTF、PSI、cgroup v2、TC/XDP、BPF LSM 和 sched_ext 等接口。内核负责提供任务调度、资源压力、控制器状态及安全事件等原始能力，用户态 Agent 负责解释观测数据、组织策略并决定是否执行控制动作。这种内核侧提供机制、用户态实现策略的划分，使项目能够在不修改通用内核主体逻辑的前提下扩展观测和管控功能。

不同 openEuler 版本及不同内核配置所具备的能力并不完全一致。例如，BTF、BPF LSM、sched_ext 或特定网络 Hook 可能因内核版本和编译选项而不可用。EulerPilot 在启动前通过 CapabilityDetector 逐项探测这些能力，并将结果转换为 available、unavailable、blocked 或降级状态。cgroup v2 作为发行内核上的稳定资源控制路径，sched_ext 等能力则作为满足条件时启用的增强路径。

2.3.2 eBPF、BTF、CO-RE 与 libbpf

eBPF 是一种在 Linux 内核受控环境中运行程序的可编程技术。eBPF 程序在加载前需要经过 Verifier 校验，并通过 tracepoint、LSM、TC、XDP、cgroup 等 Hook 接入内核事件或执行路径。程序可以借助 BPF map 在内核态保存统计信息或与用户态交换状态，也可以通过 ring buffer 等机制将事件传递给用户态。EulerPilot 使用 eBPF 完成调度事件观测、部分网络与安全策略执行，并在 sched_ext 增强路径中承载可编程调度逻辑。eBPF 的内核工作原理如图 3 所示。

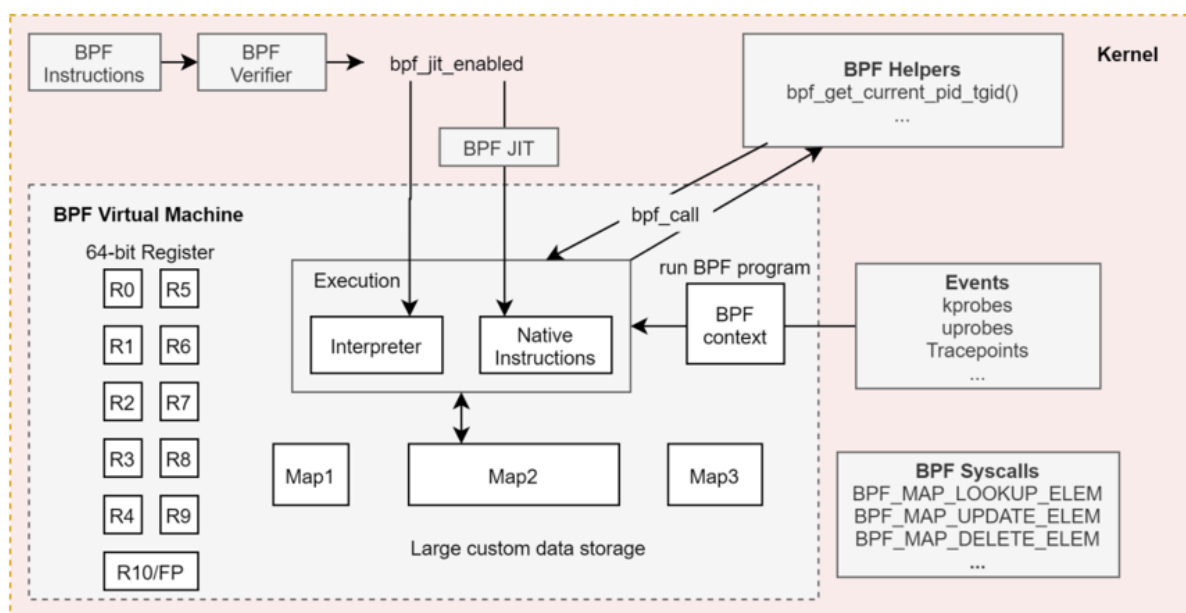


图 3 eBPF 工作原理图

eBPF 程序的执行过程如图 4 所示。用户态程序首先生成 BPF 字节码并发起加载请求，内核 Verifier 对指令安全性、控制流、内存访问和辅助函数调用等进行检查；校验通过后，程序被加载到指定 Hook，并在 kprobe、uprobe、tracepoint 或 perf event 等事件触发时执行。运行过程中，BPF 程序通过 Map 保存统计状态，并可借助 perf event 等数据通道向用户态输出结果。

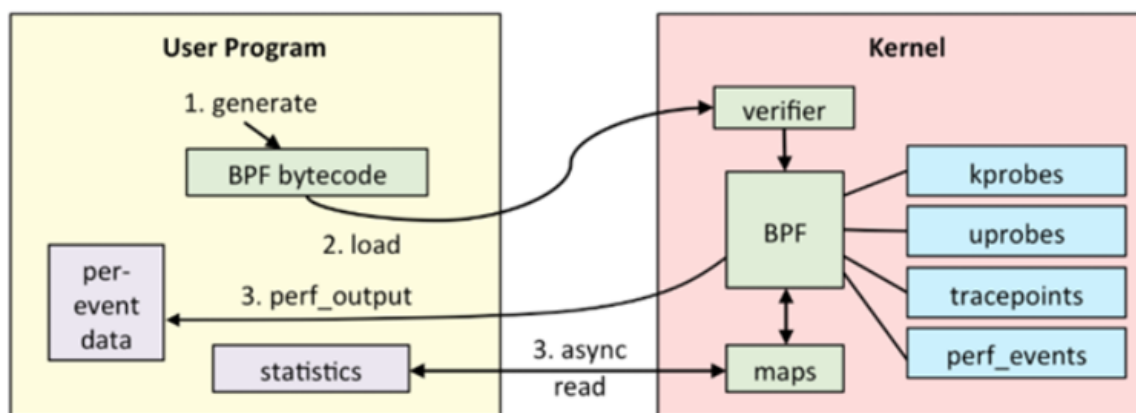


图 4 eBPF 执行过程图

BTF 用于描述内核类型信息，vmlinux.h 由目标内核 BTF 生成，为 BPF 程序提供 CO-RE 类型定义。CO-RE 使同一份 BPF 源码能够依据目标内核的实际类型布局完成重定位，降低不同内核版本之间的数据结构差异。项目使用 clang 编译 BPF 源码，通过 bpftool 生

成 skeleton，并由用户态 libbpf 完成对象加载、程序 attach、map 访问、ring buffer 消费和 link 清理。该工具链把 BPF 程序的构建、加载、通信和生命周期管理连接为统一流程。

2.3.3 PSI 与 cgroup v2 资源控制

PSI (Pressure Stall Information) 用于描述 CPU、内存和 IO 资源争用造成的任务停顿。其 some 指标表示至少部分任务因资源不足而受阻，full 指标表示相关任务整体处于停顿状态；avg10、avg60 和 avg300 反映不同时间窗口内的平均压力。EulerPilot 读取 /proc/pressure/cpu、memory 和 io，并结合任务等待增量、后台运行增量及 workload 存在性形成压力证据。PSI 在项目中主要承担压力观测职责，资源限制由 cgroup v2、sched_ext 和各类 Skill 执行^[2]。

cgroup v2 通过统一层级组织进程，并提供 CPU、内存和 IO 等资源控制接口。EulerPilot 使用 cpu.weight 和 cpu.max 调整 CPU 相对份额与带宽上限，使用 memory.high、memory.low 和 memory.max 表达内存软限制、保护值与硬上限，使用 io.weight 和 io.max 实施 IO 权重或设备级限速。执行器在写入前检查目标路径和控制文件白名单，保存原始值，写入后进行回读核验，并在失败或停止时恢复旧值。因此，PSI 负责提供压力信号，cgroup v2 负责提供稳定、可回读和可恢复的资源执行路径^[3]。

2.3.4 sched_ext 可编程调度

sched_ext 是 Linux 提供的可扩展调度类，允许通过 BPF 程序定义任务入队、选择、分发和统计等调度行为。与长期固定在内核中的调度策略相比，sched_ext 便于针对特定 workload 迭代调度逻辑，并支持动态装载与卸载。当调度器出现内部错误、可运行任务停顿或主动退出时，内核可以将任务恢复到公平调度路径，为实验性调度策略提供失效保护^[4]。

EulerPilot 将 sched_ext 作为能力探测通过后启用的增强后端。用户态 ScxExecutor 负责启动或连接 scx_eulerpilot，并将 Workload Analyzer 得到的任务分类写入 class_map，将 PsiGate 状态写入 gate_state_map；内核侧调度器依据分类和门控状态选择相应 DSQ 路径。该机制使用户态分类与内核态调度执行解耦，同时保留 cgroup v2 作为稳定主路径，避免把自定义调度器作为所有环境中的强制依赖。

2.3.5 Web Console 与证据交付技术

Web Console 采用 React、Vite 和 TypeScript 实现，用于展示 Agent 结构化状态、能力探测结果、压力与调度信息、Policy Engine 事务时间线以及 Evidence 材料。控制台通过固定接口读取系统状态和白名单任务结果，适合现场说明、日志查看和证据核对。

项目通过 CSV、JSONL、日志、run_manifest 和 report.md 保存运行结果，并使用 final_evidence_manifest.json 限定最终证据范围。collect_final_evidence.py 检查必需证据，final_quality_gate.sh 核验构建、smoke 测试、默认安全配置及无残留状态。通过运行标识、事务标识和目标标识，命令行输出、动作记录、实验结果与 Web Console 展示能够相互关联，使技术实现不仅可运行，也具备可检查、可复现和可交付的证据链。

2.4 代码原创说明

本项目组声明，本项目所提交的全部源代码均由本项目组成员独立编写完成，具备完整的自主原创性。项目中涉及的功能模块、算法实现和系统架构设计，均是在充分调研相关技术资料的基础上，自主分析、设计与开发的成果。

在开发过程中，项目参考了 openEuler 与 Linux 内核相关技术资料，以及 eBPF、cgroup v2、PSI、sched_ext、libbpf、yaml-cpp、React、Vite 和 TypeScript 等开源技术、工具与框架的公开接口和官方文档，但严格遵循相应开源协议，仅作为底层机制、依赖库或开发支撑使用，并未直接复制或抄袭其实现代码。凡涉及外部资源的部分，均在报告、许可证文件或源码注释中明确标注出处。

生产源码未直接复制 third_party/reference 中的算法实现或源文件片段，直接引用外部源码行数为 0 行。第三方组件均以系统库、构建依赖、npm 依赖或文档参考方式使用；确需保留的参考快照仅用于设计对照和可追溯说明，不进入 Makefile 生产目标。

同时，本项目中所有非原创的内容均已在正文中进行了必要的引用与说明，引用的文字、图片、算法和技术资料均符合学术规范要求，不存在任何形式的学术不端行为。

综上所述，本项目源代码及其相关文档完全符合学术规范与知识产权要求，保证不存在抄袭、剽窃等侵权行为。

此外，项目对生产依赖、系统组件和 `third_party/reference` 参考快照进行了明确区分，避免将参考材料计入原创生产实现，如表 6 所示。

表 6 第三方依赖与参考源码边界

项目	使用方式	是否进入生产源码	许可证/来源边界
libbpf	系统库，用户态 BPF 对象装载、attach、map 与 ring buffer API	链接依赖，不复制库源码	上游 libbpf; BSD-2-Clause 或 LGPL-2.1 双许可证
yaml-cpp	解析 agent.yaml 与 skills.yaml	链接依赖，不复制库源码	由系统 pkg-config 提供
libbpf-bootstrap	CO-RE、skeleton 和 ring buffer 的工程组织参考	仅 <code>third_party/reference</code> 快照	上游 BSD-3-Clause; 未进入 Makefile 生产目标
Kata-LSM-eBPF / lmp-xdp-lsm / PerfInsight PSI	LSM、XDP、PSI 设计对照	仅 <code>reference</code> 快照	各自保留上游文件与说明；生产目录重新实现
vmlinux.h	BPF CO-RE 类型定义	生成文件	由目标内核 BTF 通过 bpftool 生成，不计入手写代码量
Web 前端依赖	React、Vite、TypeScript、lucide-react、yaml	npm 依赖，不复制源码	版本范围记录于 <code>web_console/package.json</code> 与 <code>package-lock.json</code>

`sched/scx_eulerpilot.c` 与 `sched/scx_eulerpilot.bpf.c` 使用 GPL-2.0 SPDX 标识；
`bpf/workload_observer.bpf.c` 使用 GPL-2.0 OR BSD-3-Clause 标识。除文件自带 SPDX 声明外，其他项目源码遵循仓库根 `MulanPSL-2.0`。参考资料目录不应被打包成项目原创实现，也不应在代码量统计中与 EulerPilot 生产代码混合。

第 3 章 项目介绍

3.1 功能介绍

EulerPilot 面向 openEuler 混部场景，把“观测—判断—执行—验证—恢复”组织为可追踪的资源管控闭环。系统由内核观测、用户态 Agent、可插拔 Skills、资源与网络执行器、事务审计和证据交付模块共同组成。生产代码中，agent 目录约 40 个 C/C++ 头源文件、10,438 行；bpf 目录扣除生成的 vmlinux.h 后约 1,966 行；sched 目录约 847 行；web_console 约 3,969 行，承担展示和受控演示入口。

EulerPilot 的功能由用户入口、内核观测、用户态决策、Skill 编排、内核执行、状态反馈以及测试交付等环节共同形成闭环。业务负载产生的调度事件和资源压力由内核观测层采集，经 Observer Reader 进行任务身份聚合和周期增量计算后，交由 Workload Analyzer 完成负载分类。PsiGate 依据压力证据控制运行状态转换，Policy Engine 根据安全事件和策略规则生成跨 Skill 事务，二者共同形成面向 Skill Manager 的决策输入。

Skill Manager 根据能力探测结果、目标作用域和安全配置，将动作分发至 Resource Control、Scheduling、Network 和 Security 等 Skill，并通过 cgroup v2、sched_ext、TC/XDP 和 BPF LSM 等内核机制完成执行。执行结果通过控制文件回读、BPF Map 状态、AuditBus、ActionJournal 和 Runtime Status 返回用户态控制核心；当动作失败、Agent 停止或实验结束时，系统依据旧值、执行句柄和对象所有权信息完成回滚与清理。项目还通过 SP4 主验证环境、SP3 强制兼容环境、质量门禁、基准测试以及 Evidence 与 Release 流程验证功能完整性和交付可信度。系统总体运行闭环见图 5。

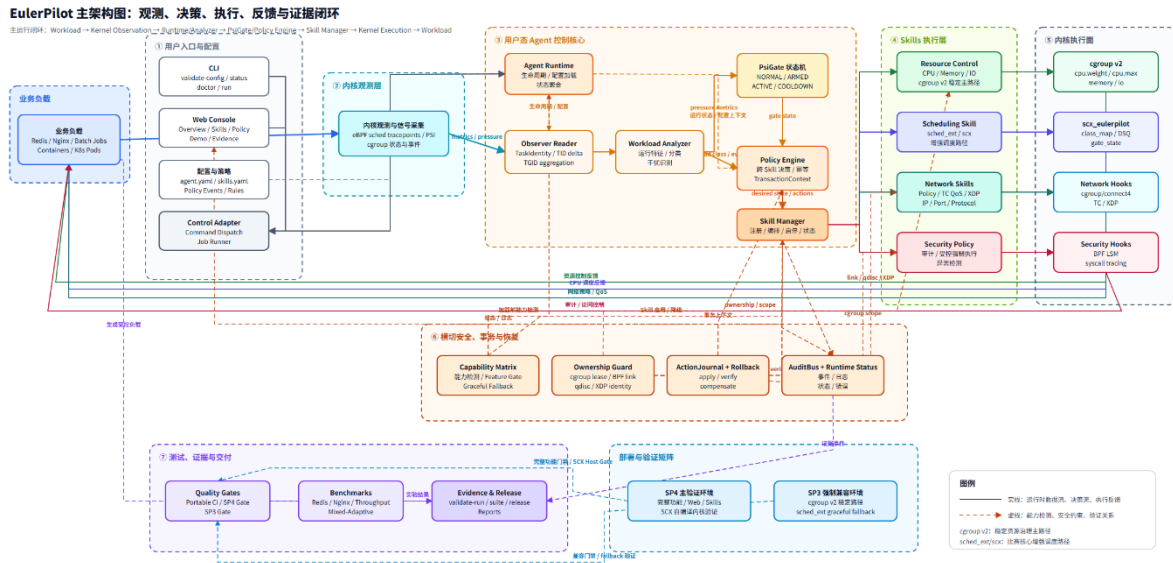


图 5 EulerPilot 观测、决策、执行、反馈与证据闭环总体架构

图 5 从端到端角度展示 EulerPilot 的主要数据流、决策流、执行流、反馈流和证据流。其中，实线表示运行时的观测数据、策略决策、内核执行和状态反馈路径，虚线表示能力探测、安全约束、对象所有权、回滚以及验证关系。cgroup v2 构成官方发行内核上的稳定资源控制路径，sched_ext 作为满足内核能力条件时启用的增强调度路径；Network 和 Security Skill 则只有在显式目标、规则和安全前提满足时才进入实际执行。测试与交付模块负责验证功能、保存结果和生成证据，不进入 Agent 热路径。

总体架构描述了系统各模块之间的运行闭环。为便于快速把握系统主线，本文给出 EulerPilot 的简化架构图，如图 6 所示。该图突出展示了系统从业务负载观测、策略决策、Skill 编排到内核执行与反馈恢复的主闭环，并保留用户入口以及横切的安全、事务与回滚约束。为了说明不同组件在系统中的层次位置，本文还从纵向功能分层的角度进行划分。EulerPilot 的分层系统架构见图 7。

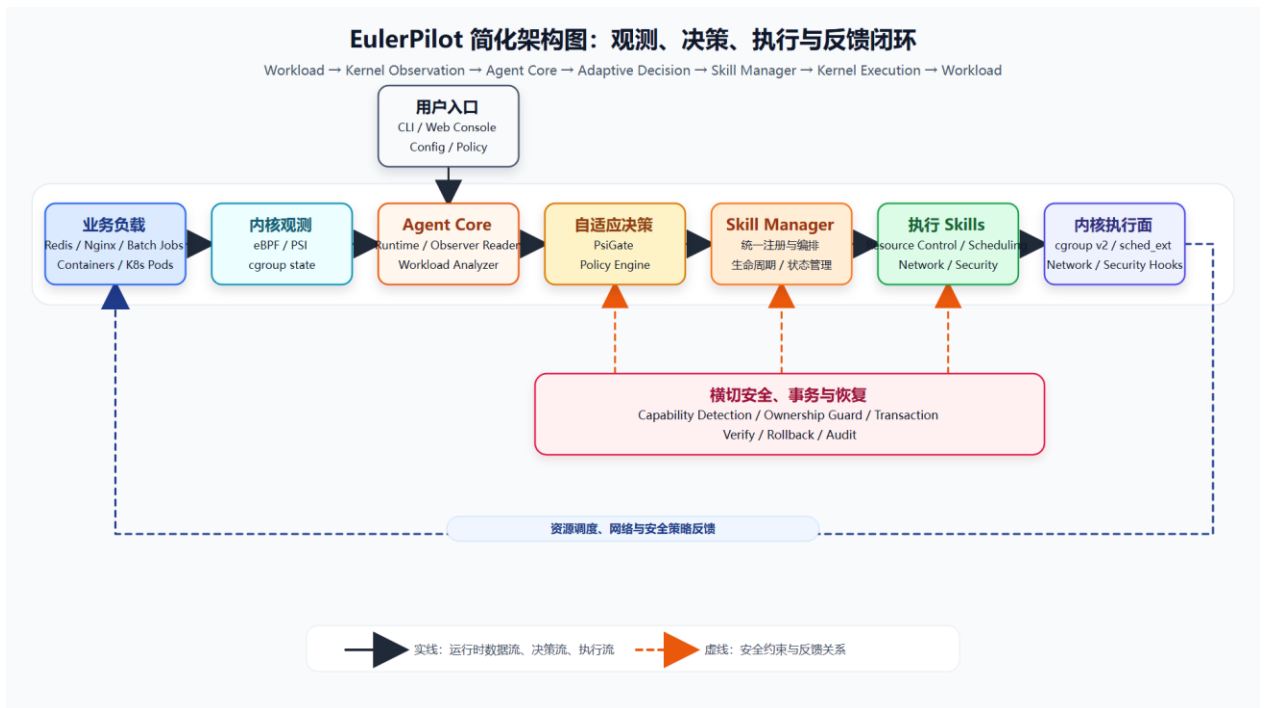


图 6 EulerPilot 简化架构图：观测、决策、执行与反馈闭环

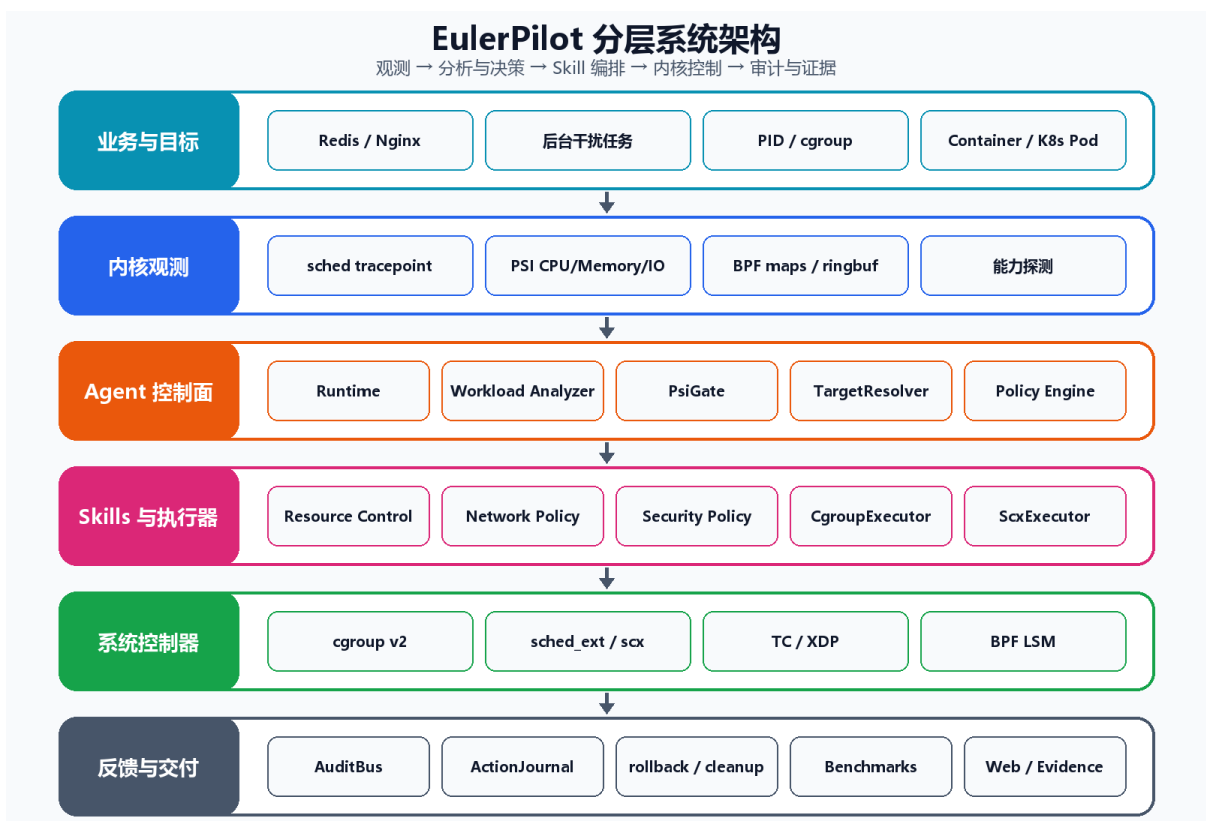


图 7 EulerPilot 分层系统架构

在上述分层结构中，各层具有不同的数据来源、处理职责和输出对象。业务与目标层用于声明待观测或待管理的工作负载；内核观测层负责采集调度、压力和安全事件；Agent 控制面完成分类、门控、目标解析与策略编排；Skills 和系统执行层负责受控地修改内核状态；反馈与交付层则保存审计、回滚、实验和 Evidence 结果。各层的主要输入、输出和职责见表 7。

表 7 分层输入输出与职责

层次	核心组件	输入	输出
内核观测层	workload_observer、PSI reader、LSM/TC/XDP hooks	调度事件、压力、系统调用、报文	任务增量、PSI、安全事件、命中计数
Agent 控制面	Runtime、Analyzer、PsiGate、TargetResolver	观测快照、环境能力、YAML	分类、GateDecision、目标、profile
Skills 能力层	resource_control、network_policy、security_policy、policy_engine	SkillSpec、RuntimeContext、触发事件	snapshot、audit、rollback
系统执行层	CgroupExecutor、ScxExecutor、BPF/link、qdisc	路径、接口、规则、动作	控制文件、BPF map、挂载状态
证据展示层	Benchmark、Evidence、Web Console	CSV、JSONL、日志、manifest	报告数据、事务时间线、白名单演示

一次完整循环从采集任务调度增量和 /proc/pressure/cpu 开始。Runtime 识别任务身份并计算本周期增量，Analyzer 形成 LatencySensitive、ThroughputBatch、BackgroundNoisy、MixedService 或 Unknown 分类；PsiGate 再用压力、延迟等待和后台运行证据决定是否允许主动干预。TargetResolver 把配置目标解析为内核对象，resource_control 或 sched_ext 执行 profile；所有变更保存旧状态，并经 AuditBus、ActionJournal 和结果目录形成证据。任何阶段失败都进入逆序恢复。

3.2 Agent Framework

3.2.1 统一 Skill 生命周期

agent/include/skill.hpp 定义 configure、probe、init、start、snapshot、rollback、stop 和 last_error。configure 解释配置但不修改系统；probe 检查能力与目标；init 准备对象和上下文；start 才进入运行态。snapshot 统一返回 available、running、state 和 evidence，供 --status --json、doctor-safe 和证据工具读取；rollback 恢复副作用；stop 终止线程并释放句柄。

SkillRegistry 保存 kind 到工厂函数的映射，SkillManager 从 YAML 创建实例，拒绝重复名称、未知 schema 和缺少依赖的配置，并按 dependencies() 拓扑排序启动。某个 Skill 的 probe、init 或 start 失败时，对已启动实例先逆序 rollback_all，再逆序 stop_all，保证依赖方先退出，避免半配置状态。

SkillRegistry 不仅负责维护 Skill 类型与工厂函数之间的映射，还向 CLI 和配置加载器提供统一的模块注册结果。程序启动前可通过--list-skills 检查当前构建版本中已经注册的 Skill，提前发现模块未编译、名称不一致或配置引用未知 Skill 等问题。当前封版版本共注册 9 个 Skill，覆盖资源控制、调度门控、网络策略、安全策略及跨 Skill 编排等功能，具体注册结果见图 8。

```
[root@cernet2 EulerPilot]# ./build/eulerpilot-agent --list-skills
network_policy
network_policy_demo
network_qos
network_xdp
policy_engine
psi_gate
resource_control
security_policy
security_policy_demo
```

图 8 Agent Skill 注册结果

在完成 Skill 注册后，各实例按照 configure、probe、init、start、snapshot、rollback 和 stop 的统一接口进入运行流程。不同生命周期阶段的主要动作、失败处理方式和可观测输出见表 8。

表 8 Skill 生命周期及故障语义

阶段	动作	失败处理	输出
configure	解析 RuntimeConfig/SkillSpec，校验字段	不触碰系统，直接拒绝	last_error、校验结果
probe	检查 BTF、cgroup v2、PSI、LSM、TC/XDP、sched_ext	unavailable 或 strict 失败	能力快照、doctor 原因
init/start	加载对象、挂载并启动工作线程	逆序 rollback + stop	running/state/evidence
snapshot	读取 map、控制文件、计数器	只读失败写入状态	状态 JSON、控制台数据
rollback/stop	恢复旧值、detach、清理 qdisc/map	记录失败并继续清理	审计与 journal

3.2.2 能力探测与安全启动

CapabilityDetector 逐项检查 BTF、ring buffer、cgroup v2、PSI、BPF LSM、TC、XDP、sched_ext 和 Kubernetes 工具链，模块的 available 状态来自实际环境探测。能力缺失时，系统根据 Skill 依赖关系返回 unavailable、blocked 或降级原因，避免把未执行探测或环境不满足误报为功能运行成功。

Agent 提供--list-skills、--doctor-safe、--doctor-skills、--validate-config、--status --json、--backend、--gate-mode、--active 和--warmup-cycles 等命令行入口。默认 configs/agent.yaml 采用 1000 ms 采样周期、dry-run 模式和 cgroup v2 后端，Prometheus exporter 默认关闭并绑定 127.0.0.1:9108。其中，doctor-safe 用于部署前的无副作用环境检查，不执行 live probe，也不修改系统控制器。

doctor-safe 执行时只读取文件系统、内核能力、依赖命令和默认安全配置，不加载 eBPF 程序，不挂载 TC/XDP 或 BPF LSM，也不创建实验 cgroup。因此，截图中的 safe-

not-probed 表示相关 Skill 未执行具有副作用的实时探测，不能解读为功能代码缺失。

doctor-safe 的能力检查及 Skill 安全诊断结果见图 9。

```
[root@cernet2 EulerPilot]# ./build/eulerpilot-agent --doctor-safe --config configs/agent.yaml

Doctor mode: safe (no BPF/TC/XDP/LSM probe)

Capability Detector
-----
+ bpf_lsm          available /sys/kernel/security/lsm
+ btf              available /sys/kernel/btf/vmlinux
+ cgroup_v2        available /sys/fs/cgroup/cgroup.controllers
! kubernetes       missing kubectl command
+ memory_reclaim   available /sys/fs/cgroup/memory.reclaim
+ psi_cpu          available /proc/pressure/cpu
+ psi_io           available /proc/pressure/io
+ psi_memory       available /proc/pressure/memory
+ ringbuf          available libbpf ringbuf requires BTF/libbpf runtime
+ sched_ext        available /sys/kernel/sched_ext
+ tc               available tc command
+ xdp              available /sys/class/net/* with generic fallback
-----

Skills Doctor
-----
! resource_control    safe-not-probed
! psi_gate            safe-not-probed
! policy_engine       safe-not-probed
! network_policy      safe-not-probed
! network_policy_demo safe-not-probed
! network_qos         safe-not-probed
! network_xdp         safe-not-probed
! security_policy     safe-not-probed
! security_policy_demo safe-not-probed
-----
```

图 9 doctor-safe 能力探测与安全诊断结果

3.2.3 任务采样、身份与分类

workload_observer 记录 pid、tgid、comm、cgroup_id、start_boottime_ns、runtime、等待、唤醒、上下文切换和迁移计数。Runtime 保存上周期基线并计算 total_wait_ns_delta、runtime_ns_delta、wakeup_count_delta、ctx_switch_count_delta 和 migrate_count_delta。计数器回退、start_boottime_ns 改变或 generation cookie 更新时，将该 PID 视为新任务并重置基线，避免 PID 复用产生虚假增量。

classify_sample 根据唤醒/等待关系、运行时长、上下文切换与迁移特征计算 latency_score、batch_score 和 interference_score，再形成五类结果。这里的“自适应”采用运行时采样、规则评分和状态机门控，不依赖机器学习训练。Unknown 表示证据不足；只有 should_manage_sample 认可且通过作用域检查的样本进入受管集合。WorkloadDecision 同时保存分数、managed_target、gate_relevant、触发原因、profile 和 ExecutionAction，能够解释“为什么判断为这类任务”“为什么允许或拒绝控制”和“实际准备执行什么动作”。

表 9 Workload 规则评分与触发标准

项目	源码口径/公式	默认阈值或判定	作用
等待比例	$r_wait = wait_ns / \max(run_ns, 1)$	$wait_ns = total_wait_ns$; $run_ns = runtime_ns$	按累计等待与运行时间计算
延迟分数	if wakeup_count > 0: $s_lat = \min(1.0, r_wait + 0.2)$ else: $s_lat = 0$	Redis、Nginx 等显式 latency 服务: $s_lat = \max(s_lat, 0.85)$	识别延迟敏感服务或类似行为
批处理分数	if run_ns > 2,000,000: $s_batch = \min(1.0, run_ns / 10,000,000)$ else: $s_batch = 0$	make、sysbench、 bench 等再结合进程语义判断	识别长运行、低唤醒的吞吐型任务
干扰分数	if migrate_count > 0: $s_intf = \min(1.0, migrate_count/4 + r_wait*0.3)$ else: $s_intf = r_wait*0.2$	stress-ng、yes、sleep 等干扰任务: $s_intf = \max(s_intf, 0.8)$	识别后台干扰或迁移频繁的任务
延迟高证据	$latency_wait_high = (wait_delta \geq T_wait)$	$T_wait = 5,000,000\ ns$	作为 PsiGate 激活证据之一
后台高证据	$background_runtime_high = (run_delta \geq T_bg)$	$T_bg = 4,000,000\ ns$	判断后台任务是否持续消耗 CPU
控制模式	Normal: no latency or no background Mixed: latency_wait_high and (cpu_psi_high or background_runtime_high) Latency: any pressure evidence	CPU PSI avg10 默认阈值=0.10; warmup 可按 P95 校准未显式设置的阈值	把单任务评分合成为全局控制意图

表 9 采用 Word 兼容的文本公式表达，不依赖 MathType 插件。式中 r_wait 表示等待/运行比例， s_lat 、 s_batch 、 s_intf 分别表示延迟、批处理和干扰分数； T_wait 、 T_bg 为触发阈值。评分结果只决定“任务像什么”和“压力是否存在”。真正写入 cgroup、sched_ext、TC/XDP 或 BPF LSM 前，还必须经过 managed_target、target scope、CapabilityDetector、Ownership Guard 和 dry-run/active 模式检查。

任务采样与分类结果会直接体现在 Agent 的周期性输出中。dry-run 模式下，Agent 持续读取任务增量并完成 workload 分类，但不会写入 cgroup、sched_ext、TC/XDP 或 BPF LSM。输出中的 class 表示 Analyzer 给出的任务类别，profile 表示当前匹配的资源策略，act 表示计划执行或观察状态，reason 则说明该任务为何进入或未进入受管集合。通过这些字段可以核对任务身份、分类结果和控制资格之间是否一致。Agent 的任务分类与 dry-run 输出见图 10。

```
[root@cernet2 EulerPilot]# ./build/eulerpilot-agent --config configs/agent-metrics.yaml --duration-s 30 --interval-ms 1000 &
curl http://127.0.0.1:9108/metrics
[1] 361841

* EulerPilot Agent
-----
config:      configs/agent-metrics.yaml
interval:    1000ms duration: 30s warmup: 0
mode:        dry-run backend:  cgroup_v2
scx binary:  auto (auto)
gate:        always-active
env:         psi:ok cg:ok scx:ok
-----

curl: (7) Failed to connect to 127.0.0.1 port 9108 after 1 ms: Couldn't connect to server
[root@cernet2 EulerPilot]# --- legend ---
mark: . UNKNOWN L LATENCY_SENSITIVE B BACKGROUND_NOISY T THROUGHPUT_BATCH
act:   + applied ~ managed - none
--- observing ---
mark name      pid class      act profile  w:weight reason
```

图 10 Agent 任务分类与 dry-run 观测结果

3.2.4 PSI Gate 与自适应阈值

PsiGate 使用 NORMAL、ARMED、ACTIVE、COOLDOWN 四态抑制抖动。默认连续 2 个激活窗口进入 ACTIVE，连续 3 个恢复窗口退出 ACTIVE，随后冷却 2000 ms。GateDecision 保存 previous_state、next_state、generation、timestamp、evidence_mask 和 profile，并可同步到 sched_ext 的 gate_state_map。

默认阈值为 CPU PSI avg10=0.10、延迟等待增量 5,000,000 ns、后台运行增量 4,000,000 ns。启用 EULERPILOT_ADAPTIVE_THRESHOLDS 且 warmup_cycles>0 时，预热样本按 P95 校准未被环境变量显式覆盖的阈值，CPU PSI 值限制在 0.001~0.50。校准值写入 WorkloadDecision，实验可区分固定阈值和自适应阈值。

在实际运行过程中，PsiGate 根据任务存在性、等待增量、后台运行增量和 CPU PSI 等证据持续更新状态。首次出现候选压力时，Gate 由 NORMAL 进入 ARMED；连续满足激活窗口后进入 ACTIVE 并应用相应 profile；压力消失达到恢复窗口后进入 COOLDOWN，冷却完成后再返回 NORMAL。运行输出中的 Gate 状态、任务类别和

profile 变化可用于核对状态转换是否与当前压力证据一致。PSI Gate 在压力场景中的状态变化见图 11。

```
[root@cernet2 EulerPilot]# ./build/eulerpilot-agent --config configs/agent.yaml --duration-s 10 --interval-ms 1000

* EulerPilot Agent
-----
config:      configs/agent.yaml
interval:    1000ms duration: 10s warmup: 0
mode:        dry-run backend:  cgroup_v2
scx binary:  auto (auto)
gate:        always-active
env:         psi:ok cg:ok scx:ok
-----

--- legend ---
mark: . UNKNOWN L LATENCY_SENSITIVE B BACKGROUND_NOISY T THROUGHPUT_BATCH
act:  + applied ~ managed - none
--- observing ---
mark name      pid class      act profile  w:weight reason
B stress-ng-cpu 392382 BACKGROUND_NOISY ~ normal_profile w:100 no-group-for-class
B sleep         393117 BACKGROUND_NOISY ~ normal_profile w:100 no-group-for-class
B stress-ng-cpu 392383 BACKGROUND_NOISY ~ normal_profile w:100 no-group-for-class
T redis-benchmark 393084 THROUGHPUT_BATCH ~ normal_profile w:100 non-target-process
L redis-server   392367 LATENCY_SENSITIVE ~ normal_profile w:100 no-group-for-class
B stress-ng-cpu 392384 BACKGROUND_NOISY ~ normal_profile w:100 no-group-for-class
B stress-ng-cpu 392382 BACKGROUND_NOISY ~ mixed_profile w:5 dry-run
B sleep         393117 BACKGROUND_NOISY ~ mixed_profile w:5 dry-run
B stress-ng-cpu 392383 BACKGROUND_NOISY ~ mixed_profile w:5 dry-run
T redis-benchmark 393084 THROUGHPUT_BATCH ~ normal_profile w:100 non-target-process
T redis-benchmark 393118 THROUGHPUT_BATCH ~ normal_profile w:100 non-target-process
L redis-server   392367 LATENCY_SENSITIVE ~ latency_profile w:1000 dry-run
B stress-ng-cpu 392384 BACKGROUND_NOISY ~ mixed_profile w:5 dry-run
B stress-ng-cpu 392382 BACKGROUND_NOISY ~ mixed_profile w:5 dry-run
B sleep         393117 BACKGROUND_NOISY ~ mixed_profile w:5 dry-run
```

图 11 PSI Gate 压力状态与 profile 切换结果

图 11 展示了 PsiGate 在实际采样周期中的状态输出。为明确各状态的进入条件、控制行为和退出条件，四状态转换规则归纳见表 10。

表 10 PSI Gate 状态机

状态	进入条件	行为	退出条件
NORMAL	初始或冷却完成	共享/普通路径	首个有效窗口进入 ARMED
ARMED	证据未达到连续窗口数	累计 activation streak	连续 2 窗口进 ACTIVE；消失回 NORMAL

状态	进入条件	行为	退出条件
ACTIVE	压力与任务证据满足	应用 latency/mixed profile 或分类 DSQ	连续 3 恢复窗口进 COOLDOWN
COOLDOWN	主动控制刚退出	恢复普通路径并排空遗留队列	2000 ms 后回 NORMAL

3.3 Resource Control Skill

3.3.1 Profile 与 CPU 控制

resource_control 将 profile 解释为 CPU、Memory、IO 或 scx 动作，并通过 SkillRuntimeContext 向 Runtime 和 Policy Engine 暴露同一组 resource_ops。normal_profile 保持默认；latency_profile 提高延迟任务保障并限制干扰源；mixed_profile 同时保护延迟任务和抑制后台任务；throughput_profile 只在 EULERPILOT_THROUGHPUT_FIRST=1 的显式实验中启用。

CPU 控制的稳定执行路径主要包括 cpu.weight 和 cpu.max。其中，cpu.weight 用于调整同一层级内不同 cgroup 的相对 CPU 份额，cpu.max 以“quota period”的形式设置 CPU 带宽上限。项目保留 cpuset.cpus 的显式实验接口，但最终封版默认设置 cpuset_control=disabled，不将动态 CPU 拓扑调整作为默认启用的完整能力。Executor 执行前校验目标路径和控制文件白名单，读取原始值后写入新参数，并通过控制文件回读确认结果；若目标不适用、格式非法或回读不一致，则返回明确错误，不通过其他控制文件替代。Analyzer 只负责选择 profile，Executor 负责将 profile 转换为具体控制参数，使分类逻辑能够在 cgroup v2 与 sched_ext 后端之间复用。

Profile 进入执行阶段后，Agent 需要同时显示当前运行模式、执行后端、任务类别、目标 profile 和动作状态，才能判断分类结果是否已经转换为资源控制请求。在 active 模式下，Agent 依据配置和 Gate 状态选择对应 profile，并由 Resource Control 将其转换为 cgroup v2 控制动作；输出中的 class、act、profile 和 reason 分别表示任务类别、当前动作、匹配策略及动作原因。CPU Profile 在 cgroup v2 主动控制模式下的运行结果见图 12。

```

* EulerPilot Agent
-----
config:      configs/agent.yaml
interval:    1000ms duration: 30s warmup: 0
mode:        active backend:  cgroup_v2
scx binary:  auto (auto)
gate:        always-active
env:         psi:ok cg:ok scx:ok
-----

--- legend ---
mark: . UNKNOWN L LATENCY_SENSITIVE B BACKGROUND_NOISY T THROUGHPUT_BATCH
act:  + applied ~ managed - none
--- observing ---
mark name      pid class      act profile      w:weight reason
B stress-ng-cpu 392383 BACKGROUND_NOISY ~ normal_profile w:100 no-group-for-class
B stress-ng-cpu 392384 BACKGROUND_NOISY ~ normal_profile w:100 no-group-for-class
B sleep         393226 BACKGROUND_NOISY - normal_profile w:100 no-group-for-class
B stress-ng-cpu 392382 BACKGROUND_NOISY ~ normal_profile w:100 no-group-for-class
L redis-server  392367 LATENCY_SENSITIVE |~ normal_profile w:100 no-group-for-class
T redis-benchmark 393211 THROUGHPUT_BATCH |~ normal_profile w:100 non-target-process
B stress-ng-cpu 392383 BACKGROUND_NOISY + mixed_profile w:5 assigned
B stress-ng-cpu 392384 BACKGROUND_NOISY + mixed_profile w:5 assigned
B sleep         393226 BACKGROUND_NOISY + mixed_profile w:5 assigned
B stress-ng-cpu 392382 BACKGROUND_NOISY + mixed_profile w:5 assigned
L redis-server  392367 LATENCY_SENSITIVE |+ latency_profile w:1000 assigned
T redis-benchmark 393211 THROUGHPUT_BATCH |~ normal_profile w:100 non-target-process
B stress-ng-cpu 392383 BACKGROUND_NOISY + mixed_profile w:5 assigned
B stress-ng-cpu 392384 BACKGROUND_NOISY + mixed_profile w:5 assigned
B sleep         393226 BACKGROUND_NOISY + mixed_profile w:5 assigned
B stress-ng-cpu 392382 BACKGROUND_NOISY + mixed_profile w:5 assigned
L redis-server  392367 LATENCY_SENSITIVE |+ latency_profile w:1000 assigned
T redis-benchmark 393211 THROUGHPUT_BATCH |~ normal_profile w:100 non-target-process

```

图 12 CPU Profile 与 cgroup v2 主动控制结果

图 12 主要用于说明任务分类、profile 选择和控制动作之间的对应关系；CPU 参数是否真正写入并生效，仍需结合控制文件回读和集成测试结果进行判断。

3.3.2 Memory 与 IO 控制

Memory 控制的稳定路径支持 `memory.high`、`memory.low` 和 `memory.max`，分别用于设置内存软限制、保护值和硬上限。项目保留 `memory.reclaim` 作为显式实验动作，最终封版默认关闭，常规自动控制路径只使用可回读、可恢复的控制文件。两项有限值同时存在时，`memory.high` 不得高于 `memory.max`；无效单位、负值和越界关系在写入前直接拒绝。Policy Engine 执行 `memory.high` 动作时复用 Resource Control 的同一校验器，跨 Skill 事务不能绕过内存边界。每个控制文件分别保存原始值，失败后按动作顺序逆向恢复。

IO 控制路径支持 `io.weight` 与 `io.max`。其中，`io.weight` 用于调整同一层级内的相对 IO 权重，`io.max` 则按照设备主次设备号配置 `rbps`、`wbps`、`riops` 或 `wiops` 等带宽及 IOPS 限

制。由于 `io.max` 可能包含多个设备行，`Executor` 在执行前保存完整原始内容，写入后只回读并核验目标设备对应的配置项，避免影响其他设备。集成测试除检查控制文件外，还通过 `fio` 或等效负载比较限制前后的实际吞吐，并保存 `rollback` 后的控制文件和性能状态，避免把“文件写入成功”直接视为 IO 控制已经有效。

`Resource Control` 通过结构化状态输出当前配置、可用能力、运行状态和执行后端。该状态同时显示 CPU、Memory 和 IO 控制能力，并明确标识动态 `cpuset` 控制在当前封版本中保持默认关闭。`Resource Control` 的状态与能力检查结果见图 13。

```
[root@cernet2 EulerPilot]# ./build/eulerpilot-agent --status --json | jq '.cpuset_control, .skills[] | select(.name=="resource_control")'
{
  "name": "resource_control",
  "available": false,
  "running": false,
  "state": "created",
  "evidence": {
    "backend": "resource_control_adapter",
    "cpu_max": "enabled",
    "io": "enabled",
    "memory": "enabled",
    "mode": "enforce",
    "reason": "ok",
    "scheduler_backend": "cgroup_v2",
    "scx_binary": "/usr/local/bin/scx_eulerpilot",
    "scx_binary_executable": "true",
    "scx_binary_source": "PATH"
  }
}
```

图 13 Resource Control 状态与能力检查

3.3.3 路径边界、回读和回滚

`CgroupExecutor` 对路径做 `canonical/weakly_canonical` 归一化，只允许白名单根目录内的 `cgroup`；文件名只允许已实现控制项。每次动作记录 `old_value`、`new_value`、`target` 和 `result`。部分动作成功、后续失败时按逆序恢复。`rollback` 消除系统副作用，`stop` 终止循环并释放资源；即使某个恢复项失败，仍继续清理其他项并把失败写入审计。回滚完成后还需再次读取目标控制文件，并核对测试 `cgroup`、进程归属和相关内核对象是否恢复到执行前状态；只有回读确认的结果才记录为 `ROLLED_BACK`。

`Resource Control` 的回滚过程以执行前保存的文件值和目标状态为依据，不写统一默认值。测试过程中，`ActionJournal` 记录每项控制动作的目标、原始值、新值和执行结果；当后续动作失败、`Agent` 停止或测试进入 `cleanup` 阶段时，系统按照相反顺序恢复已经完成的操作，并再次回读目标控制文件。`Resource Control` 的回滚执行和状态恢复结果见图 14。

```
[root@cernet2 EulerPilot]# sudo tests/integration/test_resource_control.sh
sudo tests/integration/test_resource_control_io.sh
make: Nothing to be done for 'agent'.
[INFO] background hog pid=362414
[INFO] resource_control integration result saved to results/resource_control/integration-20260727-093745
make: Nothing to be done for 'agent'.
[INFO] background hog pid=362613 io_device=253:0 baseline_time=0.037s
[INFO] resource_control IO integration result saved to results/resource_control/io-20260727-093757
[root@cernet2 EulerPilot]# sudo scripts/rollback.sh
[rollback] scx binary: /usr/local/bin/scx_eulerpilot
[Rollback] restore cgroup parameters and stop active sched_ext scheduler if configured.
[Rollback] done.
```

图 14 Resource Control 集成测试与回滚结果

图 14 展示了 Resource Control 在实际执行中的回滚结果。由于 CPU、Memory、IO 和 sched_ext 所操作的控制文件或内核对象不同，各控制域需要采用不同的写前检查、结果核验和恢复方法，具体执行边界见表 11。

表 11 Resource Control 执行边界

控制域	文件/对象	写前检查	验证与恢复
CPU	cpu.weight、cpu.max、 cpuset.cpus 显式实验，默认关闭	路径、格式、文件存在	回读；逐文件恢复旧值
Memory	memory.low/high/max、 memory.reclaim	数值/单位、 $high \leq max$	回读；按原值恢复
IO	io.weight、io.max	设备号、键和值	回读目标行；恢复多行内容
sched_ext	class_map、gate_state_map	后端能力、PID/类别、 map 可用	stats/gate-status；删除或 恢复 map 项

3.4 TargetResolver

Resource Control、Network Policy 和 Security Policy 在执行动作前，都需要先将配置中的用户侧目标转换为唯一、可验证的内核对象。TargetResolver 负责统一处理 cgroup 路径、进程 PID、容器标识和 Kubernetes Pod 等不同目标形式，并输出后续执行器所需的 cgroup 路径、cgroup ID、进程 PID 或宿主机侧网络接口。该模块只负责目标解析和边界校验，不根据模糊名称猜测目标，也不会目标无法唯一确认时继续执行系统写入。

3.4.1 cgroup、PID 与容器

对于显式 cgroup 目标，TargetResolver 首先对输入路径进行规范化，并确认解析结果位于 `/sys/fs/cgroup` 层级内且目标目录真实存在。对于 PID 目标，解析器读取 `/proc/<pid>/cgroup`，获得该进程在 cgroup v2 统一层级中的实际路径，再取得对应的 cgroup ID。若进程已经退出、cgroup 目录不存在或路径无法解析，则返回明确失败原因，不进入后续控制阶段。

对于容器目标，配置可以使用 `container_name` 或 `container_id` 进行引用。TargetResolver 根据当前环境选择 Docker、Podman、iSulad 或 containerd/CRI 工具获取容器的 init PID，再复用 PID 目标的解析路径，从 `/proc/<pid>/cgroup` 确定实际 cgroup。该过程不直接根据容器 ID 拼接 systemd 目录名称，避免不同容器运行时、cgroup 驱动和命名规则造成错误映射。容器未运行、查询结果冲突、init PID 缺失时，解析器返回 `blocked` 或 `unavailable`。

通过将 cgroup、PID 和容器三类引用最终统一到实际 cgroup 路径，后续 Resource Control 和 Security Policy 可以复用同一目标检查与作用域控制逻辑，而不需要分别处理不同运行时的目录命名差异。

3.4.2 Kubernetes Pod 与 host veth

Kubernetes Pod 目标可以通过 `namespace`、`pod_name`、`pod_uid` 或 `label selector` 进行描述。TargetResolver 首先通过 Kubernetes 接口确认唯一 Pod 及其 UID，再取得容器 ID，并通过 CRI 定位容器进程 PID 和对应 cgroup。使用 `label selector` 时，只有匹配结果能够唯一确定目标才继续解析；出现零匹配或多匹配时返回失败原因，不任意选择其中一个 Pod。

当 Network Policy 需要作用于 Pod 网络接口时，TargetResolver 进入目标 Pod 对应的网络命名空间，读取容器侧接口的 `iflink`，再在宿主机接口中匹配对应的 `ifindex`，确定 Pod 的 host veth。若 Pod 使用 `hostNetwork`，系统默认返回 `pod-host-network-not-allowed`，不将宿主机主网络接口作为普通 Pod 接口处理，以降低误挂载 TC 或 XDP 程序的风险。

解析结果统一保存 `target_type`、原始目标引用、`cgroup_path`、`pid`、`cgroup_id`、`host_veth`、`host_ifindex` 和 `reason` 等字段。不同 Skill 可以根据自身需求读取其中的资源目标或网络目标，同时在审计记录、状态 JSON 和测试结果中保留相同的目标标识。

只有配置中显式声明的实验接口，或由目标 Pod 唯一反查得到的 host veth，才允许交给 TC 或 XDP 执行器。若接口在执行前消失、Pod 与接口映射不唯一、目标使用 hostNetwork，或 kubectl、CRI 等工具链不可用，解析结果应返回 blocked 或 unavailable 及相应原因，禁止扩大扫描范围或根据接口名称猜测目标。

不同目标类型虽然最终都需要转换为内核对象，但其解析入口、依赖工具、输出字段和失败条件并不相同。为便于比较 cgroup、PID、容器、Pod 及 Pod 网络接口的解析过程，各类目标的主要解析链路和失败边界见表 12。

表 12 TargetResolver 解析链路

目标	解析链路	输出	失败边界
cgroup	规范化→根目录/存在性校验	path/id	越出根目录、目录不存在
PID	/proc/<pid>/cgroup	PID + cgroup	进程退出、层级不可解析
Container	运行时→init PID→/proc	容器 PID + cgroup	未运行、冲突、PID 失效
Pod	kubectl→UID/容器 ID→CRI PID	Pod cgroup	selector 多匹配、CRI 不可用
Pod 网络	netns iflink→host ifindex	host veth	hostNetwork、映射不唯一

TargetResolver 的解析结果是后续系统动作的必要前提，但解析成功本身并不代表控制已经执行。Resource Control 仍需校验控制文件和参数，Network Policy 仍需检查接口所有权与规则范围，Security Policy 仍需确认作用域和内核能力。目标在实际执行前还会重新核验，以避免 Pod 重建、容器退出、PID 变化或 host veth 更新造成陈旧引用。

3.5 sched_ext 调度增强

sched_ext 为 EulerPilot 提供可编程调度增强路径，用于将 Agent 产生的任务分类和 PsiGate 状态传递到内核调度策略中。该路径与 cgroup v2 稳定后端并行存在，项目启动也不依赖 sched_ext；只有当目标内核、BTF、BPF 环境和调度器构建产物均满足要求时才允

许启用。若能力条件不成立，系统应明确输出不可用原因，并继续保留 cgroup v2 或只读观测路径。

3.5.1 能力条件和 Map 联动

sched_ext 启用前需要依次检查目标内核是否配置并开放 sched_ext 接口、/sys/kernel/sched_ext 是否存在、BTF 与 BPF 基础能力是否可用，以及当前源码对应的调度器二进制和 BPF 对象是否能够正常构建。任何一项必要条件缺失时，Scheduling Skill 均返回 unavailable 及具体原因，并回退到 cgroup v2 或 observe-only 模式，不将能力缺失误报为调度器已经运行。

sched_ext 启用前需要依次检查目标内核是否配置并开放 sched_ext 接口、/sys/kernel/sched_ext 是否存在、BTF 与 BPF 基础能力是否可用，以及当前源码对应的调度器二进制和 BPF 对象是否能够正常构建。任何一项必要条件缺失时，Scheduling Skill 均返回 unavailable 及具体原因，并回退到 cgroup v2 或 observe-only 模式，不将能力缺失误报为调度器已经运行。

能力检查通过后，ScxExecutor 将受管 PID 及其任务类别写入 class_map，并将 PsiGate 的 state、generation、updated_at_ns 和 evidence_mask 写入 gate_state_map。用户态调度器通过--stats、--gate-status 和--gate-set 等只读或受控接口核验 Map 状态，相关对象统一固定在/sys/fs/bpf/eulerpilot/scx_eulerpilot/v1 路径下。通过这种方式，Agent 负责分类和门控决策，内核侧 sched_ext 程序负责按照 Map 中的结果执行调度，两者不需要共享未受控的用户态命令接口。

在完成能力检查和 Map 初始化后，sched_ext 才进入实际调度阶段；后续还需要通过 DSQ 命中计数、Gate 状态、调度统计和退出恢复结果判断增强路径是否真正运行。

3.5.2 DSQ 策略和失效保护

内核侧 BPF struct_ops 创建 latency、batch、background 和 shared 四类 DSQ。当前策略为延迟敏感、批处理和后台类别分别设置不同时间片，并通过后台饥饿保护避免低优先级任务长期得不到运行机会。PsiGate 处于 ACTIVE 状态时，新入队任务根据 class_map 进入相应分类 DSQ；处于 NORMAL、ARMED 或 COOLDOWN 状态时，新任务进入 shared 路

径。dispatch 阶段仍会尝试排空状态切换前遗留的分类队列，避免 Gate 状态变化后任务滞留。

当 class_map 中不存在目标 PID、类别值非法、gate_state_map 不可读或调度状态不满足条件时，任务统一进入 shared fail-safe 路径，避免错误提升为延迟敏感类别。用户态调度器异常退出或主动停止后，内核 sched_ext 框架恢复默认调度类；Agent 停止时还需释放已固定的 Map 和相关对象，避免调度器进程退出后残留错误状态。

sched_ext 路径验证需要同时观察调度器加载状态、class_hits、enqueue、dispatch、running、shared fallback、Gate 状态和退出恢复结果。上述指标只能证明可编程调度路径及门控联动已经生效，不能单独证明业务性能得到提升。当前正式结果中部分 sched_ext 实验出现退化，因此本模块的封版结论限定为“调度增强路径可运行、可观测、可降级和可恢复”，不宣传其对所有 workload 具有稳定正向收益。

为验证 sched_ext 增强路径的实际可用性，测试首先检查目标系统是否存在 /sys/kernel/sched_ext 接口，再从当前源码构建 scx_eulerpilot，随后运行限定场景的 smoke 测试，核对调度器加载、任务运行、状态统计和退出恢复过程。该测试用于确认 sched_ext 控制链能够在 SP4 自编译内核上运行，不作为普遍性能提升证据。sched_ext 的能力检查与 smoke 验证结果见图 15。

```
[root@cernet2 EulerPilot]# test -d /sys/kernel/sched_ext && echo sched_ext-ok
sched_ext-ok
[root@cernet2 EulerPilot]# scripts/build_scx_eulerpilot.sh
[FAIL] KERNEL_SRC is required; pass the SP4/OLK kernel source path explicitly
[root@cernet2 EulerPilot]# sudo bench/redis/run_redis_scheduled_ext_smoke.sh
sudo bench/nginx/run_nginx_scheduled_ext_smoke.sh
[INFO] Redis scheduled_ext smoke output: /root/EulerPilot/results/reports/redis-scx-smoke-20260727-100530
[INFO] redis port=6384 stress_workers=2 clients=16 requests=20000
368247:C 27 Jul 2026 10:05:31.060 # WARNING Memory overcommit must be enabled! Without it, a background save or replication may fail under low memory condition. Being disabled, it can also cause failures without low memory condition, see https://github.com/jemalloc/jemalloc/issues/1328. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then reboot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
[INFO] Redis scheduled_ext smoke complete.
[INFO] Nginx scheduled_ext smoke output: /root/EulerPilot/results/reports/nginx-scx-smoke-20260727-100645
[INFO] Nginx scheduled_ext smoke complete.
```

图 15 sched_ext 能力检查与 smoke 验证结果

3.6 Network Policy Skill

Network Policy Skill 面向连接控制、出口流量治理和报文处理等网络场景，分别通过 cgroup connect4、TC QoS 和 XDP 三类机制执行。不同网络能力使用独立的 BPF 程序、Map、link 或 qdisc 对象，但均复用 TargetResolver 提供的目标解析结果，并遵循显式目

标、接口白名单、对象所有权记录和可回滚清理原则。网络功能默认保持关闭或审计状态，只有在目标接口、策略规则和安全前提明确时才允许进入实际执行。

3.6.1 connect4、TC QoS 与 XDP

cgroup connect4 路径将 BPF 程序挂载到目标 cgroup 的 socket connect hook，并根据 cgroup_id、目标 IPv4 地址、端口和策略模式匹配连接请求。audit 模式只记录命中事件并允许连接继续执行，enforce 模式仅对显式命中的规则返回拒绝结果。规则 Map 保存目标范围和策略内容，事件与命中计数用于核对规则是否真正作用于指定 cgroup；未配置目标、规则不匹配或 scope 外的进程保持原有网络行为。

TC QoS 路径在目标接口的 egress 方向挂载 classifier，并通过 TBF qdisc 设置 rate、burst 等限速参数。执行前，Network Skill 记录目标接口、原有 qdisc 和 filter 状态；执行后，通过 tc 状态、规则句柄和实际传输结果核对限速是否生效。测试结束、动作失败或 Agent 停止时，系统按照对象所有权恢复原 qdisc，或仅删除本次测试创建的 TC 对象，避免覆盖接口上原有的外部网络配置。

为验证连接策略和出口限速路径能够在隔离网络环境中运行，集成测试首先创建并检查 lab veth 连通性，再分别启动 Network Policy 和 Network QoS 测试，核对 Skill 注册、目标接口识别、连接策略执行及 fallback 行为。测试完成后调用对应 cleanup 脚本，撤销本次验证创建或挂载的网络对象。Network Policy 与 TC QoS 的集成测试结果见图 16。

```
[root@cernet2 EulerPilot]# sudo tests/integration/test_network_policy.sh
sudo tests/integration/test_network_qos_tc.sh
sudo tests/integration/test_network_xdp.sh
=== NetworkPolicySkill integration test ===
PASS: formal network_policy skill is registered
=== NetworkQos TC integration test ===
PASS: network_qos skill is registered
PASS: lab veth baseline connectivity works
=== NetworkXDP integration test ===
PASS: network_xdp skill is registered
PASS: lab veth baseline connectivity works
[root@cernet2 EulerPilot]# sudo scripts/cleanup_network_policy_demo.sh || true
sudo scripts/cleanup_network_qos_tc.sh || true
sudo scripts/cleanup_network_xdp_demo.sh || true
```

图 16 Network Policy 与 TC QoS 集成测试结果

XDP 路径用于在隔离 veth 或由 TargetResolver 唯一解析出的 Pod host veth 上执行报文级 pass/drop 验证。项目采用 generic XDP 模式，并支持 ICMP、TCP、UDP 以及 UDP tuple

等规则类型；规则 Map 保存 rule_id、匹配字段和命中计数，测试通过对照流量及 Map 计数核对规则效果。cleanup 阶段需要 detach 本次挂载的 XDP 程序并清理对应 Map 项；对于测试脚本创建的临时 veth，可以依据所有权记录删除，而 Kubernetes 或 CNI 管理的真实 Pod host veth 只能解除 EulerPilot 自身的挂载，不能删除接口。

三类网络能力虽然执行机制不同，但均要求目标来源明确。cgroup connect4 只接受显式解析的目标 cgroup；TC QoS 和 XDP 只接受配置中允许的 lab 接口，或由 TargetResolver 根据目标 Pod 唯一反查得到的 host veth。Network Skill 默认拒绝管理网卡、默认路由接口、当前 SSH 连接相关接口以及无法确认所有权的外部对象；目标接口在执行前消失、映射不唯一或安全条件不满足时，应返回 blocked 或 unavailable，禁止扩大扫描范围或尝试操作相似名称的接口。

cgroup connect4、TC QoS 和 XDP 分别作用于连接请求、出口流量和报文接收路径，其挂载对象、验证方式和恢复方法并不相同。三类网络子能力的执行机制、验证对象及清理边界见表 13。

表 13 Network Policy 子能力

子能力	挂载/机制	验证对象	恢复
connect4	cgroup/connect4	IPv4 connect allow/deny 与命中	销毁 link、清规则 map
TC QoS	egress classifier + TBF	速率、字节、qdisc 状态	恢复或删除 qdisc/filter
XDP	generic XDP	协议/tuple drop-pass 与计数	detach、清 map、按所有权清接口

Network Policy 的测试通过并不意味着可以对任意接口直接启用网络控制。正式执行还必须满足目标唯一、接口在允许范围内、规则可解析、原状态能够保存以及 cleanup 路径可验证等条件。对于真实 Pod 网络对象，EulerPilot 只撤销自身附加的 BPF、TC 或 XDP 对象，不修改 Kubernetes 和 CNI 对接口的所有权。

3.7 Security Policy Skill

Security Policy Skill 面向文件访问、程序执行、网络连接、进程调试、Capability 使用和凭据变化等安全行为，通过 BPF LSM 完成作用域内的策略执行，并结合系统调用 tracepoint 补充审计和异常事件。该模块区分 audit 与 enforce 两种运行模式：audit 模式只记录命中事件，不改变原有系统行为；enforce 模式只有在规则、目标和 cgroup scope 均明确时才执行阻断。安全事件还可以进入异常聚合器，并作为 Policy Engine 跨 Skill 事务的触发来源。

3.7.1 LSM、规则与异常聚合

Security Policy 通过 BPF LSM Hook 对安全敏感操作进行作用域内检查，当前覆盖 file_open、bprm_check_security、socket_connect、ptrace_traceme 和 capable 等路径。文件规则根据路径或路径前缀、读写类型和 cgroup scope 进行匹配；执行规则根据程序路径或前缀匹配；网络规则根据目标 IPv4 地址、端口和作用域匹配；Ptrace 与 Capability 规则则以目标 cgroup 为主要限制条件。

每条规则均明确区分 audit 和 enforce 模式。audit 模式仅生成事件并允许原操作继续执行，适合能力核验和规则观察；enforce 模式在规则显式命中且目标位于允许 scope 内时返回拒绝结果。scope 外进程保持原有行为，未提供目标范围或无法可靠解析作用域的全局 enforce 配置应在启动前被拒绝，避免安全规则误作用于整台主机。

除 LSM 执行路径外，模块还通过 execve、openat、connect 和 ptrace 等系统调用 tracepoint 补充审计信息。tracepoint 事件用于记录调用主体、目标对象、时间戳和事件上下文，并为异常聚合和跨 Skill 策略提供稳定输入；LSM 命中计数与用户态事件数量分别保存，便于识别事件消费丢失或过滤差异。

异常规则按照 event、window、threshold、comm、path 和 target 等条件维护滑动时间窗口。聚合器先移除窗口外的历史命中，再判断当前计数是否达到阈值；满足条件时生成 anomaly 事件，并记录触发事件、时间窗口、阈值、命中数量和过滤条件。阈值范围限定为 1~100000，空规则、非法模式、无效端口和不可解析 scope 在 configure 阶段直接拒绝，不进入运行态。

3.7.2 加载、消费与清理

在初始化阶段，Security Policy 加载对应的 BPF skeleton，将通过校验的文件、执行、网络、Ptrace、Capability 及异常规则写入 BPF Map，并按照当前配置挂载所需 Hook。各 BPF link 的程序类型、目标范围和所有权均由当前 Skill 实例记录。若任一必要 Hook 加载或 attach 失败，模块停止继续初始化，并逆序释放已经成功挂载的 link 和 Map 资源。

进入运行态后，事件消费线程持续读取 ring buffer，将内核事件转换为稳定的 JSONL 字段，并写入 AuditBus。异常聚合器根据规则窗口处理事件，达到阈值时生成带有 rule_id、目标、命中数量和证据字段的 anomaly。ring buffer 暂时没有事件不视为模块故障；只有加载、attach、poll 或事件解析失败时，才更新 last_error 并向运行状态输出明确原因。

Agent 停止、Skill 启动失败或测试进入 cleanup 阶段时，系统先停止事件消费线程，再解除本实例创建的 BPF link，并清理对应规则 Map。清理过程只撤销 EulerPilot 自身加载的对象，不处理其他程序创建的 LSM、tracepoint 或 BPF 资源。对于当前普通用户态场景下命中尚不稳定的 deep credential hooks，证据中保留 no-userland-hit-observed 等边界说明，不把“程序可加载”描述为“所有凭据事件均已稳定命中”。

文件、执行、网络、Ptrace、Capability 和异常聚合所处理的系统行为不同，因此各类规则需要使用不同的匹配字段、执行方式和安全边界。Security Policy 的主要规则模型及作用范围见表 14。

表 14 Security Policy 规则模型

类型	字段	行为	边界
文件	path/prefix、读写、cgroup	审计/拒绝 open	scope 内生效、外部允许
执行	exec path/prefix、cgroup	审计/拒绝 exec	字面前缀、退出 detach
网络	dst IPv4/port、cgroup	审计/拒绝 connect	仅显式目标
Ptrace/Capability	cgroup、capability	审计/拒绝检查	scope 外对照

类型	字段	行为	边界
异常聚合	event/window/threshold + filters	生成 anomaly	窗口、阈值、组合过滤

Security Policy 产生的审计事件和 anomaly 不仅用于安全结果查看，也可以作为 Policy Engine 的事务触发输入。但 Security Policy 本身只负责规则匹配、事件生成和作用域内执行，不直接调用 Resource Control 或 Network QoS。跨 Skill 动作必须由 Policy Engine 根据白名单配置创建独立 transaction_id，并通过 ActionJournal 记录执行与回滚过程。

3.8 Policy Engine 与证据链

Policy Engine 用于把 Security Policy 产生的异常事件或其他 Skill 事件转换为受控的跨 Skill 动作，使资源、网络和安全能力在统一事务边界内协同执行。该模块只通过已注册的 Resource 和 Network 操作接口调用白名单动作，并使用 transaction_id、AuditBus 和 ActionJournal 记录触发来源、执行过程、结果验证及回滚状态。评审者可以从原始事件出发，追踪一次策略联动产生的全部系统副作用及其恢复结果。

3.8.1 跨 Skill 事务

Policy Engine 订阅 Security Policy 输出的 JSONL 事件，也可以消费其他已注册 Skill 产生的结构化事件。策略配置使用 YAML 描述 policy_id、触发事件类型、过滤条件、target_ref 和有序 actions。在 configure 阶段，系统校验策略标识、动作类型、目标引用、控制参数以及动作与已注册操作接口之间的对应关系；配置中不能指定任意系统文件、可执行程序或 shell 命令。

当事件满足策略的触发类型和过滤条件时，Policy Engine 为本次策略执行创建唯一 transaction_id，并将原始事件的 event_id 记录为 trigger_event_id。其中，trigger_event_id 表示事务由哪条事件触发，transaction_id 则表示本次系统副作用的完整边界。即使同一异常事件同时匹配多条策略，各策略也分别创建独立事务，避免其中一个事务回滚时撤销其他策略已经完成的动作。

事务中的动作按照 YAML 声明顺序执行。每个动作在真正修改系统状态前，先通过 TargetResolver 重新解析目标，再调用已经注册的 resource_ops 或 network_ops。Resource Control 动作可以修改 cpu.max、memory.high 等允许的资源控制项，Network 动作可以在限定的 lab 接口或唯一解析出的 host veth 上执行 TC QoS 等白名单操作。Policy Engine 只负责动作编排，不绕过各 Skill 已有的路径校验、参数检查、对象所有权和安全作用域。

每项动作执行前均保存原始控制值、BPF link、qdisc 句柄或其他恢复信息，执行成功后记录为 APPLIED。若后续动作失败，Policy Engine 按照与执行顺序相反的方向调用恢复接口。例如，事务已经成功写入 memory.high，但随后 TC 挂载失败时，系统先撤销本次创建的网络对象，再恢复原始内存控制值。所有恢复步骤及其结果均写入 ActionJournal，只有经过回读或对象状态核验的动作才记录为 ROLLED_BACK。

为验证 Policy Engine 能够将安全事件、资源控制和网络治理组织在统一事务中，测试首先完成策略配置校验，随后分别执行单轮联动验证和连续 10 轮稳定性验证。测试同时覆盖正常动作链和失败回滚场景，用于检查跨 Skill 动作在执行失败时能否按照事务边界恢复。测试结果显示，单轮验证和连续 10 轮验证均完成，失败回滚路径能够正常执行。Policy Engine 跨 Skill 联动与回滚验证结果见图 17。

```
[root@cernet2 EulerPilot]# ./build/eulerpilot-agent --validate-config configs/policy_engine_security_network_resource.yaml
EulerPilot config valid: configs/policy_engine_security_network_resource.yaml
[root@cernet2 EulerPilot]# sudo tests/integration/test_policy_engine_security_network_resource.sh
make: Nothing to be done for 'agent'.
make: Nothing to be done for 'security-policy'.
clang -g -O2 -target bpf -D__TARGET_ARCH_x86 -D__BPF__ -I. -I./bpf -c bpf/network_qos_tc.bpf.c -o build/network_qos_tc.bpf.o
[INFO] policy_engine security-network-resource success iteration 1/1
[INFO] policy_engine failure rollback case
[INFO] policy_engine security-network-resource result saved to results/policy_engine/security-network-resource-20260727-095653
[root@cernet2 EulerPilot]# sudo tests/integration/test_policy_engine_security_network_resource.sh --repeat 10
make: Nothing to be done for 'agent'.
make: Nothing to be done for 'security-policy'.
make: Nothing to be done for 'network-qos-tc'.
[INFO] policy_engine security-network-resource success iteration 1/10
[INFO] policy_engine security-network-resource success iteration 2/10
[INFO] policy_engine security-network-resource success iteration 3/10
[INFO] policy_engine security-network-resource success iteration 4/10
[INFO] policy_engine security-network-resource success iteration 5/10
[INFO] policy_engine security-network-resource success iteration 6/10
[INFO] policy_engine security-network-resource success iteration 7/10
[INFO] policy_engine security-network-resource success iteration 8/10
[INFO] policy_engine security-network-resource success iteration 9/10
[INFO] policy_engine security-network-resource success iteration 10/10
[INFO] policy_engine failure rollback case
[INFO] policy_engine security-network-resource result saved to results/policy_engine/security-network-resource-20260727-100056
[root@cernet2 EulerPilot]#
```

图 17 Policy Engine 跨 Skill 联动与回滚验证结果

3.8.2 AuditBus、ActionJournal 与门禁

AuditBus 负责把各 Skill 产生的运行事件统一转换为 JSONL 记录。主要字段包括 timestamp、event_id、skill、policy_id、rule_id、transaction_id、trigger_event_id、mode、target、operation、action、result、severity 和 evidence。统一字段使 Security Policy 的触发事件、Policy Engine 决策以及 Resource 和 Network 动作能够按照相同标识进行关联，而不需要依赖不同模块各自的非结构化终端日志。

ActionJournal 面向实际产生系统副作用的操作，记录 action_id、动作类型、目标对象、原始值、新值、执行句柄和当前状态。单项动作通常经历待执行、APPLIED 和 ROLLED_BACK 等状态；当事务包含多个动作时，评审者可以依据 transaction_id 定位本次事务的全部 ActionJournal 记录，再逐项检查执行顺序、失败位置及恢复结果。

项目建立统一的最终证据清单，用于明确认可的证据范围、条目分组、必要性和摘要信息。证据收集过程严格依据预先确定的清单执行，不递归扫描结果目录，也不从多个实验结果中选择对项目更有利的数据。历史结果、临时结果或未经认可的材料即使仍被保留，也不会自动进入最终证据集合，交付结论因此具备明确来源和一致口径。

在交付阶段，系统首先核对最终证据条目的完整性和状态，检查是否存在必需材料缺失或结果警告；随后对构建、单元测试、配置诊断、稳定性测试、默认安全设置、证据完整性和环境清理状态进行统一验收。任何必要检查未通过时，均保留明确的失败状态和原因，不通过报告整理过程掩盖问题。

Policy Engine、AuditBus、ActionJournal 和 Evidence manifest 共同形成从“事件触发”到“动作执行”，再到“结果恢复和交付验收”的证据链。上述数据可以通过 CLI、JSONL 文件、结果目录和 release manifest 直接核对；证据展示只负责呈现已经保存的状态和结果，不重新生成策略结论，也不替代控制文件回读。

3.9 Web Console

Web Console 是 EulerPilot 面向使用者和评审人员提供的证据展示与受控演示界面，用于集中查看运行环境、Agent 状态、调度与压力信息、eBPF 扩展能力、跨 Skill 事务以及最终 Evidence。页面展示的状态和结果来自 Agent 输出、审计记录和已保存的验证材料，主要服务于现场讲解、复现检查和结果追踪。

3.9.1 技术栈、定位与页面结构

Web Console 前端采用 React、TypeScript 和 Vite 构建，后端通过轻量级服务提供系统信息、Agent 状态、Evidence、事务事件和受控任务等数据。控制台默认只监听本机回环地址，远程使用时通过 SSH 隧道映射到本地浏览器访问，不直接将具有系统操作能力的服务暴露到外部网络。封版阶段已在 SP4 主验证环境完成控制台启动和页面访问验证。

控制台按照使用目标划分为总览、Skills 与 Agent、调度与 PSI、eBPF 扩展能力、Policy Engine 事务时间线以及 Evidence 与现场演示等页面。总览页用于确认环境和交付状态；Skills 与 Agent 页用于查看模块注册、运行状态和安全诊断；调度与 PSI 页用于展示压力门控及性能结果；eBPF 扩展页用于呈现资源、网络和安全能力；事务时间线用于追踪跨 Skill 动作；Evidence 与现场演示页则用于查看最终证据、运行白名单任务和观察实时日志。

页面加载时分别读取系统环境、Agent 状态、Skill 诊断、Evidence、任务状态和事务记录。对于读取失败、能力不可用或证据缺失的内容，界面显示明确的错误或警告状态，不使用默认值将其渲染为成功。页面只展示已经保存或能够定位来源的数据，不因刷新页面而生成新的性能结果，也不以界面颜色代替控制文件回读、事件记录或质量门禁结论。

总览页面是 Web Console 的统一入口，用于集中显示主机环境、内核版本、代码版本、Agent 状态、质量门禁和 Evidence 完整度。通过该页面，使用者可以首先确认当前展示内容对应的验证环境和候选版本，并快速识别成功、警告、不可用或失败状态。Web Console 总览页面见图 18。

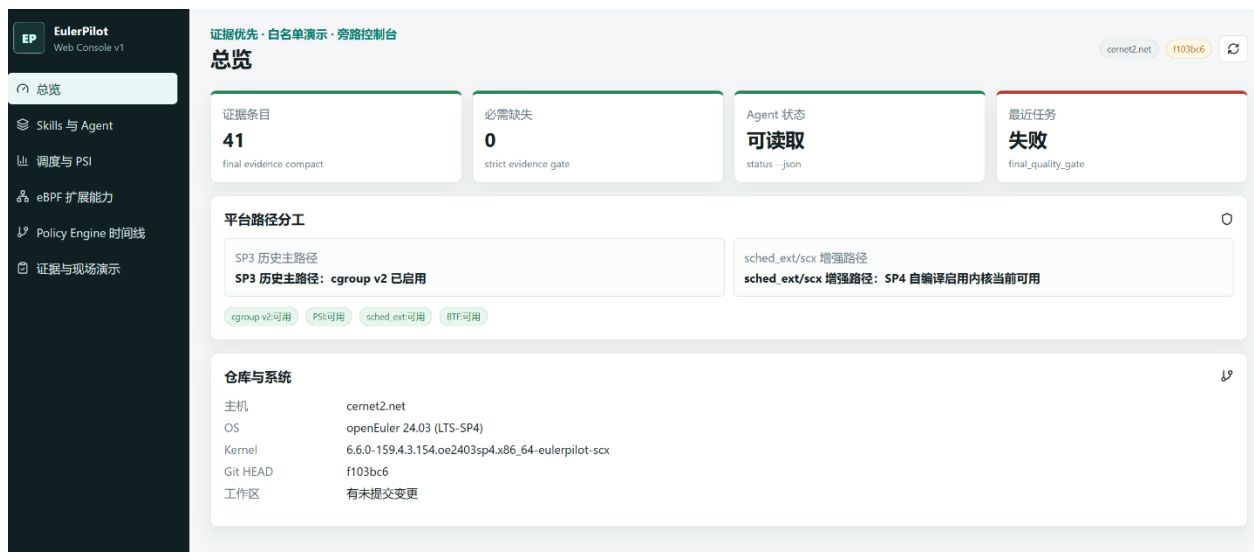


图 18 Web Console 总览页面

Skills 与 Agent 页面用于展示当前版本注册的 Skill、Agent 结构化状态、安全诊断结果和配置检查信息。页面将 available、running、运行状态和诊断原因分别显示，使使用者能够区分“未执行实时探测”“环境能力缺失”“模块尚未启动”和“模块运行失败”等不同情况。

Skills 与 Agent 页面见图 19。

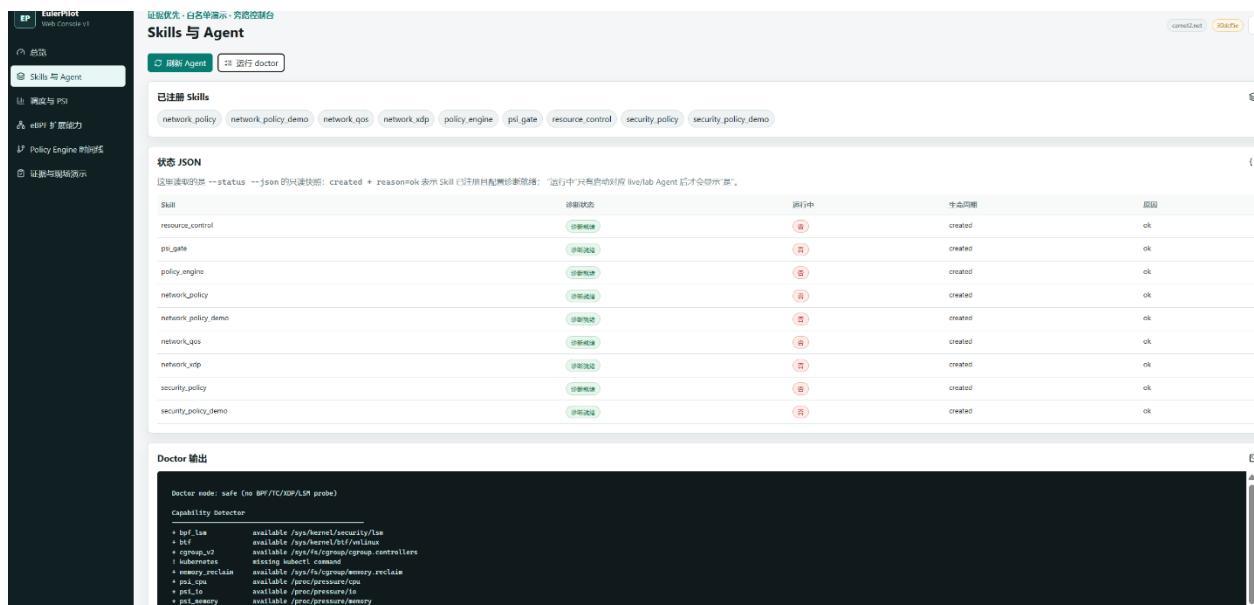


图 19 Web Console Skills 与 Agent 页面

调度与 PSI 页面用于展示负载场景、压力门控状态、调度后端和已经认可的性能图表。页面中的数据用于辅助观察 Redis、Nginx、sched_ext 和 PsiGate 等场景的状态与结

果，但不重新运行正式实验，也不将不同测试环境或不同被测版本的数据混合展示。调度与 PSI 页面见图 20。

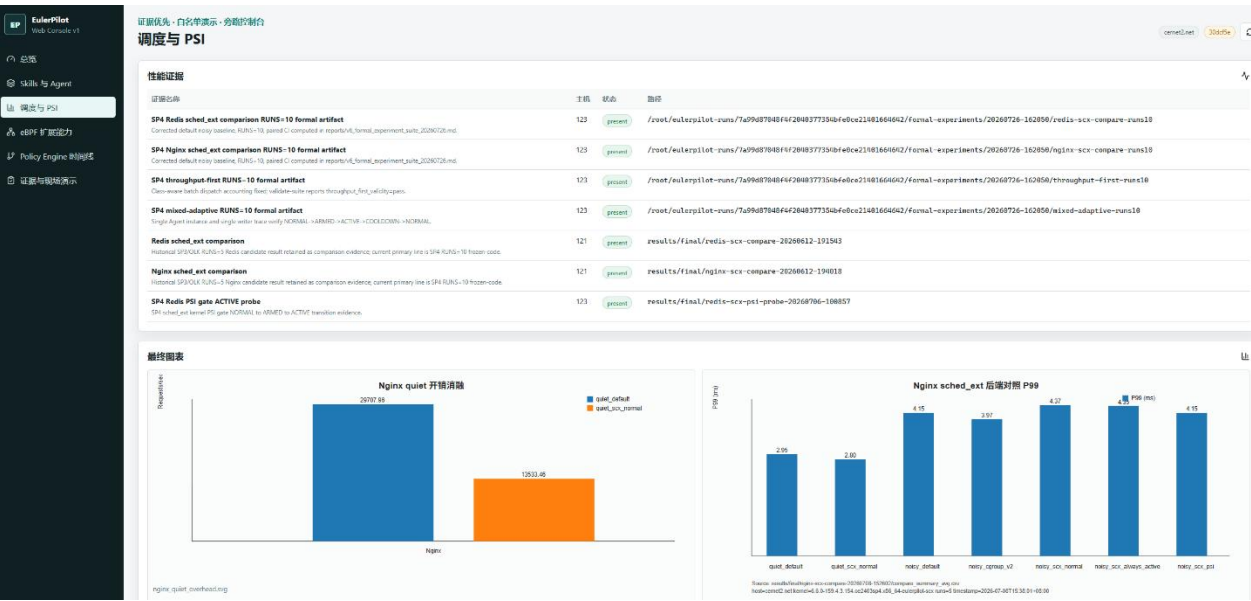


图 20 Web Console 调度与 PSI 页面

eBPF 扩展能力页面按照资源控制、网络策略和安全策略等功能类别展示模块状态、验证结果和证据入口。该页面用于说明各扩展能力是否完成配置、能力探测或集成验证，界面不提供任意 BPF 程序、目标接口或安全规则的自由输入。eBPF 扩展能力页面见图 21。

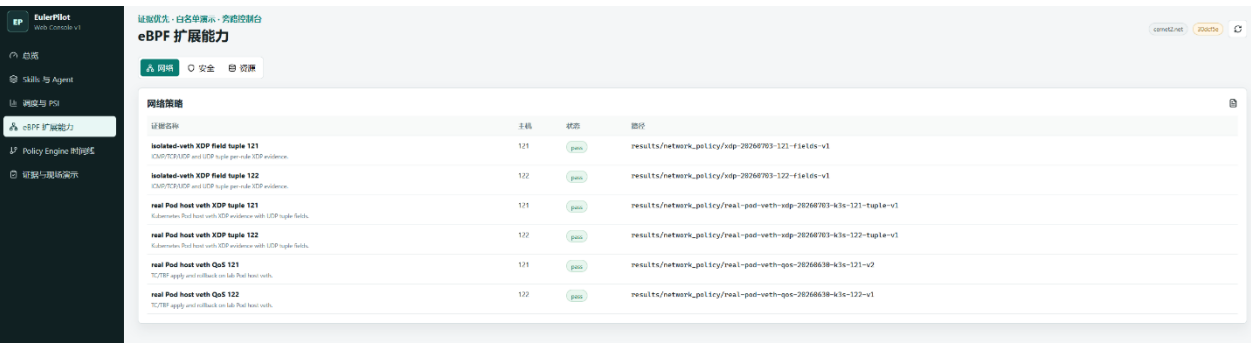


图 21 Web Console eBPF 扩展能力页面

Policy Engine 事务时间线按照 transaction_id 组织跨 Skill 联动过程，将触发事件、策略决策、资源动作、网络动作和恢复状态显示在同一事务上下文中。使用者可以通过时间

线核对一次策略联动的动作顺序和最终状态；当没有可用事务记录时，页面明确提示无数据，避免生成示例成功记录。Policy Engine 事务时间线见图 22。



图 22 Web Console Policy Engine 事务时间线

Evidence 页面用于集中展示最终证据的分类、完整性和状态，使使用者能够查看各类验证材料是否存在、是否被认可以及是否存在缺失或警告。页面仅展示预先确定的最终证据范围，不递归选择历史目录中的其他结果，也不根据页面展示状态自动修改证据结论。Web Console Evidence 页面见图 23。

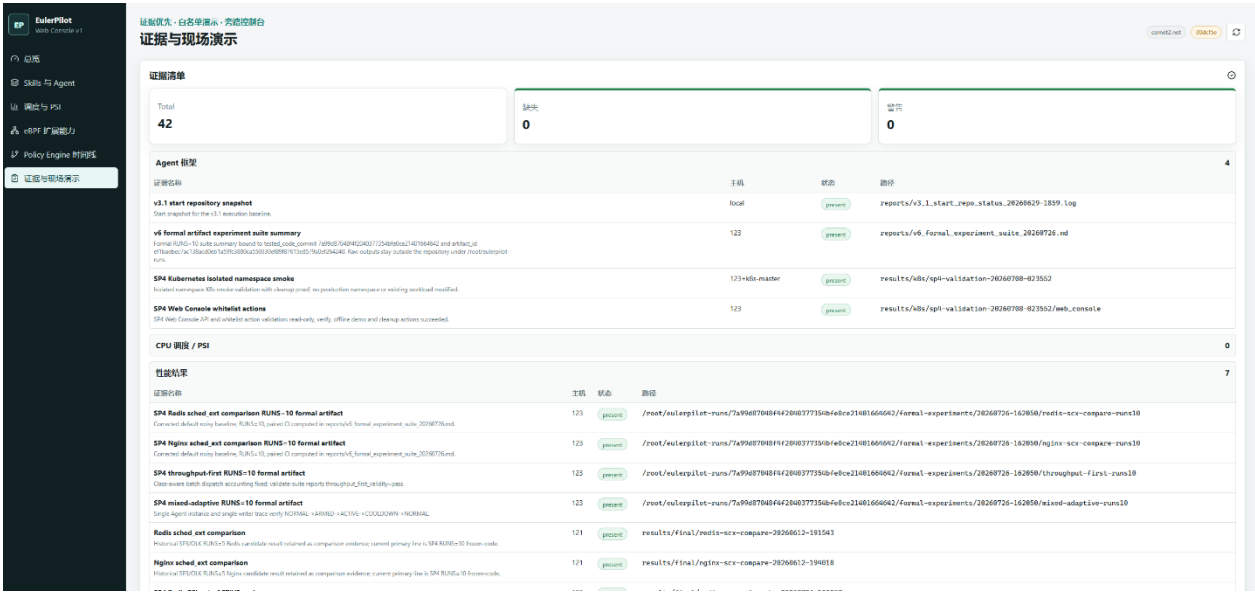


图 23 Web Console Evidence 页面

图 18 至图 23 分别展示了 Web Console 的环境总览、Agent 诊断、调度分析、eBPF 扩展、事务追踪和 Evidence 查看功能。不同页面所读取的数据、承担的展示任务和结果判读边界并不相同，各页面的功能定位见表 15。

表 15 Web Console 页面功能

页面	主要展示内容	功能定位	判读边界
总览	主机、内核、版本、Agent、质量门和 Evidence 完整度	确认环境与交付状态	不以单一卡片替代完整门禁
Skills 与 Agent	Skill 注册、运行状态、安全诊断、配置检查	模块状态与环境诊断	未探测不等于功能缺失
调度与 PSI	工作负载、PsiGate、调度状态和性能图表	展示已有调度与实验结果	不重新生成正式实验数据
eBPF 扩展能力	资源、网络和安全能力状态	集中查看扩展模块结果	不允许指定任意程序或目标

页面	主要展示内容	功能定位	判读边界
事务时间线	触发事件、策略动作、 执行和恢复状态	跨 Skill 事务追踪	以事务标识和事件 记录为依据
Evidence 与现场 演示	最终证据、白名单任 务、任务日志和清理入 口	证据查看及受控演示	变更动作需授权和 确认

3.9.2 API、白名单任务与安全控制

Web Console 后端将只读查询与系统变更操作严格分离。只读接口用于获取系统环境、Agent 状态、Skill 诊断、Evidence、事件记录和任务状态，可以并行访问；变更接口只允许启动预先审核的演示、实验或清理任务，不能接收任意命令、任意脚本内容或任意系统路径。

每个可执行任务均预先定义用途、参数范围、运行时间、输出上限、安全说明和风险边界。任务启动前，后端检查任务类型和参数是否位于允许范围内；演示、实验和清理等可能修改系统状态的任务共用单一变更任务槽，已有变更任务运行时拒绝启动新的变更任务，防止多个脚本同时修改 cgroup、TC、XDP、BPF 或其他系统对象。

具有系统副作用的请求必须通过访问授权和二次确认。二次确认信息与当前任务、参数和会话绑定，并具有较短有效期，使用后立即失效。浏览器中的确认提示只承担交互作用，真正的安全检查由后端完成；即使绕过前端直接发送请求，也仍需满足授权、任务白名单、参数校验和确认绑定等条件。

任务启动后，控制台通过持续日志流展示当前输出、运行状态和结束结果。新打开的页面可以先获取任务状态和最近日志，再继续接收后续输出；网络连接中断只表示日志显示中断，不能将任务自动标记为成功或失败。任务完成、超时或取消后，后端记录退出状态，并终止该任务派生的进程组，降低子进程遗留和资源未清理的风险。

控制台根据数据读取、任务启动、日志查看和任务终止等不同用途划分接口类别，并对具有系统副作用的操作实施更严格的授权和确认要求。主要接口类别及其安全控制见表 16。

表 16 Web Console 主要 API

接口类别	访问方式	主要用途	安全控制
系统与环境查询	只读请求	获取主机、内核、版本和能力状态	固定数据范围、只读采集
Agent 状态与诊断	只读请求	读取运行状态、Skill 和安全诊断	固定命令类型、执行超时
Evidence 与事务查询	只读请求	读取证据摘要、图表和事务记录	限定认可范围和读取数量
事件读取	只读请求	按照 Skill 或数量读取最近事件	过滤参数和数量上限
白名单任务启动	变更请求	启动演示、实验或清理任务	访问授权、参数校验和二次确认
实时日志	持续读取	查看任务输出和运行状态	限定任务标识和输出上限
任务取消	变更请求	终止正在运行的任务及其派生进程	访问授权和任务绑定确认

Web Console 通过上述页面和接口将 Agent 状态、Skill 能力、事务记录和 Evidence 集中呈现。页面中的任务状态、日志和事务结果依赖后端运行状态以及各模块保存的结构化记录。下一节将说明 Runtime、执行器和 Policy Engine 之间的数据对象、操作契约及任务生命周期。

3. 10 模块协作接口与运行状态

前述各节分别介绍了 Agent、资源控制、目标解析、调度增强、网络策略、安全策略和 Policy Engine 的独立功能。本节说明这些模块在系统运行过程中如何传递观测数据、调

用受控操作、维护内核对象所有权、记录事务状态并对外输出统一运行信息。模块协作的重点在于让同一轮观测、判断、执行和恢复使用一致的目标标识、状态语义和证据来源。

3. 10. 1 Runtime 主循环的数据组织

Runtime 负责将启动参数与配置内容统一转换为采样周期、运行时长、预热轮数、运行模式、执行后端、门控模式和监控状态等运行参数。用户在启动时显式指定的选项具有更高优先级，未明确指定的项目才采用配置值或系统默认值，避免配置文件在无提示的情况下覆盖现场运行参数。

每个采样周期依次完成压力信息读取、任务观测、周期增量计算、负载分类、触发条件构建、门控判断、profile 分配、执行器调用和运行状态导出。触发条件不仅包含全局 CPU 压力，还包括延迟任务与后台任务是否存在、可参与门控的任务数量、延迟等待增量、后台运行增量以及当前阈值，避免仅凭单一全局压力值对所有任务实施控制。

同一周期内的任务样本、分类结果、压力证据、门控代次和执行时间共同构成一次完整判断。下一周期开始时，系统重新计算任务增量和目标状态，不沿用上一周期的瞬时分类结果。该设计可以降低任务退出、PID 复用、压力变化或目标重建造成的陈旧决策。

启用自适应阈值时，预热阶段只收集有效样本用于阈值校准，不执行主动控制；完成校准后才进入正常判断流程。正常结束、收到停止请求或运行过程出现异常时，Runtime 均沿统一退出路径停止状态导出、回滚已启动 Skill 并释放调度器和内核对象，避免在信号处理过程中直接执行复杂清理操作。

Runtime 在观测、分类、门控和执行阶段使用不同的数据对象传递信息，使各模块能够明确区分原始样本、分析结果、压力条件、状态决策和实际动作。主要运行时数据对象及其生产、消费关系见表 17。

表 17 Runtime 核心数据对象

运行时对象	主要内容	产生阶段	使用阶段
任务样本	任务身份、等待、运行、唤醒、上下文切换和迁移增量	Observer 与 Runtime	Analyzer 与门控模块

运行时对象	主要内容	产生阶段	使用阶段
负载决策	任务类别、评分、profile、管理原因和计划动作	Analyzer 与 Runtime	执行器、状态输出
触发上下文	压力、任务存在性、当前阈值和高压证据	Runtime	PsiGate
门控决策	前后状态、代次、时间和证据掩码	PsiGate	资源控制与调度执行器
执行动作	执行后端、目标对象、资源参数和执行结果	执行器	审计、命令行和 Web Console

3. 10. 2 Resource 操作契约

Resource Control 将目标解析、当前状态读取、控制动作执行和状态恢复组织为统一的受控操作。Runtime 根据周期性 profile 调用这些操作，Policy Engine 在事件触发事务中也复用同一入口。因此，两条执行路径共享目标白名单、参数校验、控制文件回读、审计记录和回滚逻辑，不会形成一条严格受控、另一条可以任意写入系统的旁路。

调用方只能选择已经实现并注册的资源操作，不能指定任意控制文件或系统路径。每项操作在执行前重新核对目标是否仍然存在、控制项是否被允许、参数是否满足范围关系，并读取执行前状态。若目标已经处于期望值，系统记录为无变化而不重复写入；若目标在解析后消失，则返回目标不可用，不继续执行。

对同一 profile 或事务中的多个控制动作，系统分别记录目标、原始值、期望值、执行结果和恢复状态。这样既能判断整个 profile 是否成功，也能定位某一个控制项是否已经写入、失败或恢复，避免使用单一成功标记掩盖部分执行状态。

Resource Control 向外输出配置状态、可用状态、运行状态、执行后端、目标解析结果和最近动作证据。最终判定来自控制文件回读、集成验证和认可的 Evidence；任何展示层都不得重新扫描任意 cgroup 或仅凭界面状态推断控制成功。

3.10.3 故障可见性与降级矩阵

系统将功能状态区分为不可用、被阻断、执行失败和仅观测四类。不可用表示运行环境缺少必要能力；被阻断表示目标、配置或安全前提不满足；执行失败表示模块已经尝试初始化或执行，但出现实际错误；仅观测表示观测和判断路径正常，但系统不执行写入动作。上述状态分别进入状态快照和诊断输出，避免使用单一布尔值掩盖原因。

`sched_ext` 不可用时，系统保留 `cgroup v2` 和观测路径；`BPF LSM` 不可用时，`Security Policy` 不启动 `LSM` 执行，但资源控制、网络功能和其他诊断仍可运行；`Kubernetes` 工具链不可用时，只阻断 `Pod` 及 `host veth` 目标，不影响显式 `cgroup`、`PID` 和实验接口。能力缺失应通过 `status`、`doctor-safe` 和 `Evidence` 明确显示，不把不可用能力包装成通过。

在普通运行模式下，非必要能力缺失可以按照预定路径降级；在验收模式下，被定义为必要的能力、配置或证据缺失会升级为非零失败。事务执行过程中若后续动作失败，则本事务进入回滚流程，其他独立事务和只读审计仍保持可用。

不同能力缺失或运行故障对系统功能的影响范围不同，系统需要在保留可用路径的同时，对受影响功能输出明确状态和原因。主要降级行为见表 18。

表 18 能力缺失与降级行为

缺失/故障	受影响功能	保留功能	外显状态
<code>sched_ext</code> 不可用	分类 DSQ 与 <code>scx map</code>	<code>cgroup v2</code> 、观测、其他 Skills	<code>unavailable + fallback reason</code>
<code>BPF LSM</code> 不可用	<code>Security enforce/LSM audit</code>	<code>Resource</code> 、 <code>Network</code> 、 <code>trace/诊断</code>	<code>security unavailable</code>
<code>Kubernetes</code> 工具缺失	<code>Pod/host veth</code> 目标	显式 <code>cgroup</code> 、 <code>PID</code> 、 <code>lab netdev</code>	<code>target blocked</code>
目标运行中消失	本次动作	其他目标与只读状态	<code>target-unavailable</code> ，不写入
<code>Web API</code> 局部失败	对应页面卡片	其他只读接口和已保存证据	错误横幅、无伪成功

缺失/故障	受影响功能	保留功能	外显状态
事务后续动作失败	本事务提交	审计与其他事务	ROLLING_BACK/ROLLED_BACK

3.10.4 配置校验与验收入口

配置校验采用系统级结构检查与模块级参数检查两级方式。系统级检查确认运行周期、运行模式、调度后端、监控设置和 Skill 列表等总体结构；模块级检查再验证各自的必需字段、枚举值、数值范围、目标引用和依赖关系。未声明字段、非法参数或缺失依赖应在启动系统功能前被明确拒绝。

Resource Control 需要明确 profile、目标和允许的资源控制项；Network Policy 需要明确能力类型、接口来源和规则范围；Security Policy 需要区分审计与执行模式，并提供可解析的作用域；Policy Engine 中的动作必须能够映射到已经注册的资源或网络操作。错误输出使用稳定原因描述，便于测试和验收定位，避免只报告通用配置失败。

配置校验、基础环境诊断、Skill 能力探测和运行状态查询承担不同职责。配置校验用于回答配置结构是否正确；基础环境诊断用于检查机器是否具备安全启动条件；Skill 能力探测用于判断当前配置中的模块能否启动；运行状态用于说明各模块当前正在执行什么。四类入口不能相互替代。

集成测试在限定实验目标上执行实际写入、效果核验、回滚和 cleanup，用于证明功能链路可运行；最终质量门禁则综合检查构建、默认安全、稳定性、Evidence 完整性和无残留状态，用于判断当前版本是否满足交付条件。

验收应先完成配置校验和基础诊断，再执行限定目标的集成测试，最后运行最终质量门禁。若目标解析或基础能力尚未通过就直接运行变更实验，失败原因将难以归属；若集成测试未完成回滚和清理，最终门禁应阻止相关结果进入认可证据。命令行、结构化状态、事件记录和结果目录使用统一目标标识、策略标识、事务标识和运行标识，使各类证据能够交叉核对。

项目从配置检查、环境诊断、模块探测、运行状态、功能测试到最终门禁建立了分阶段验收流程，各入口是否修改系统及其主要检查范围见表 19。

表 19 配置诊断与验收入口

验收入口	是否修改系统	主要检查范围	典型用途
配置校验	否	配置结构、字段、范围和依赖	提交前与部署前检查
基础环境诊断	否	内核、文件系统、依赖命令和默认安全状态	首次部署与兼容性核验
Skill 能力探测	原则上仅探测	各 Skill 能力、目标和启动前提	定位不可用或被阻断原因
运行状态查询	否	运行模式、Gate、Skill 状态和 Evidence	自动化采集与 Web 展示
集成测试	是，限定实验目标	实际执行、效果、回滚和 cleanup	功能验收
最终质量门禁	包含受控测试	构建、安全默认、稳定性、证据和无残留	交付验收

通过统一数据对象、受控操作契约、对象所有权、事务状态和分阶段验收入口，EulerPilot 将各独立功能组织为能够协同运行、定位故障和恢复状态的系统。下一节将从完整运行过程出发，说明观测、分析、门控、目标解析、动作执行、事务恢复和最终清理如何形成端到端闭环。

3.11 端到端功能闭环

前述模块只有在观测证据、目标范围、执行动作和恢复结果保持一致时，才能构成可验证的系统闭环。EulerPilot 的端到端运行过程由观测、分析、门控、目标解析、动作执行、结果核验和状态恢复共同组成。系统首先采集任务调度行为和资源压力，按照同一采样周期计算任务增量并识别工作负载类型；随后结合延迟任务、后台竞争任务和压力证据

判断是否需要进入主动控制状态。只有任务身份、压力条件和管理范围同时满足要求时，系统才继续解析目标并生成受控动作。

目标解析完成后，系统根据当前能力和策略选择相应执行路径。资源控制路径通过 `cgroup v2` 调整 CPU、内存或 IO 参数，调度增强路径在能力满足时更新任务分类和门控状态，网络与安全路径则只对显式目标和白名单规则执行操作。每项动作执行前保存原始状态，执行后通过控制文件、内核对象状态或事件记录进行核验，不能仅根据命令返回值判断成功。

恢复是完整控制流程的一部分。当压力消失、Agent 正常停止、Skill 启动失败、跨 Skill 事务执行失败或测试进入清理阶段时，系统均根据执行前保存的信息恢复已经修改的控制项，并再次核对恢复结果。若 `sched_ext`、BPF LSM、Kubernetes 工具链或某项网络能力不可用，系统只关闭或阻断受影响的路径，同时保留 `cgroup v2`、只读观测和其他可用功能，避免单项能力缺失导致整个系统失效。

为保证各阶段结果能够交叉核对，观测、判断、执行和恢复过程使用统一的目标标识、策略标识、事务标识和运行标识。命令行状态、结构化事件、动作记录和实验结果均以这些标识关联。Policy Engine 时间线用于展示事务内的动作顺序，Evidence 清单用于汇总已经认可的验证材料；展示结果本身不替代控制文件回读、回滚状态和最终交付门禁。系统端到端闭环见图 27。

在现场演示或验收过程中，应先确认运行环境、Agent 状态和基础能力，再查看 Skill 状态及 Evidence；随后根据现场条件选择只读展示或限定实验目标的联动任务，最后核对任务日志、事务时间线、回滚状态和清理结果。对于暂不具备执行条件的能力，应明确显示不可用或被阻断原因，不通过临时修改配置扩大操作范围。该顺序能够使评审者区分已有正式证据、现场新产生的执行结果以及当前环境暂不可用的功能。

第 4 章 实验测试

4.1 测试说明

4.1.1 测试分层与可复现策略

测试分为静态构建、单元测试、功能集成、真实运行时/Pod、性能对照和最终门禁六层。Windows 当前工作区只用于审阅源码与生成文档；涉及 cgroup、BPF、TC/XDP、LSM、sched_ext 或 Kubernetes 的测试证据均来自仓库保存的 openEuler 运行结果。本文引用对应结果目录、summary 和门禁日志，不把未在当前 Windows 环境重跑的 Linux 集成测试描述为本地新执行结果。

项目测试从构建与配置检查逐步扩展到单元、集成、性能、前端展示和最终门禁，并为每层保存对应证据，如表 20 所示。

表 20 测试层级与证据

层级	代表入口	主要断言	证据形式
构建/配置	make agent; --validate-config; --doctor-safe	依赖、schema、默认安全与能力	退出码、控制台日志
单元测试	test_runtime_policy.cpp; test_skill_registry.cpp	分类、阈值、PID 复用、日志、registry	assert/测试二进制
集成测试	tests/integration/*.sh（24 个脚本）	CPU/Memory/IO、 Network、Security、Policy Engine	summary.txt、 JSONL、前后状态
性能测试	bench 与 tests/benchmark（7 个脚本）	RPS、P99、ops/s、限速误差、开销	CSV、 run_manifest、 report.md
Web 测试	backend/test/*.test.js（3 个文件）	action 白名单、job、HTTP 安全	Node test 结果
最终门禁	scripts/final_quality_gate.sh	构建、smoke、默认禁用、 证据存在、无残留	SP4 29/29 P0、100 轮 smoke、5 轮

层级	代表入口	主要断言	证据形式
			doctor-safe; SP3 10/10 compatibility gate

4. 1. 2 验证环境

SP4/123、SP3/121 和 122 三套环境承担主验证、强制兼容与历史对照等不同角色，如表 21 所示。SP4/123 是主开发和性能实验环境；SP3/121 是比赛要求的强制兼容交付环境；122 仅保留为历史 OLK/sched_ext 对照，相关结论不能跨环境混用。

表 21 实验环境与角色

环境	系统/内核	用途	结论边界
SP3 / 121	openEuler 24.03 LTS SP3 发行环境	cgroup v2 主闭环、构建、兼容、回滚、安全扩展 smoke	比赛发行环境兼容主线；sched_ext 不可用时应优雅降级
OLK-6.6 / 122	openEuler SP3 + 6.6.0-olk66-scx	历史 sched_ext 对照与双机功能证据	保留为历史增强对照，不作为最终 release 来源
SP4 / 123	openEuler 24.03 LTS SP4 + 官方源码自编译启用 sched_ext 内核	2026-07-24 frozen-code 性能、K8s、Web Console、质量门禁	不声称 SP4 发行默认内核直接支持 sched_ext

openEuler 官方文档提供 24.03 LTS SP3 的发布说明、安装与运维入口^[5]。项目在 README、技术报告和最终 evidence 中记录 121、122、123 的角色；当前 formal artifact 绑定 tested_code_commit=7a99d87048f4f2040377354bfe0ce21401664642、artifact_id=ef1baebec7ac138acd0eb1a59fc3880ca550330ef89f87615e8579a0ef264240，避免把后续源码变更与旧实验混用。

4.1.3 有效性门禁

- 多组性能测试采用多轮运行与平衡轮换，保存 run_order 和 run_manifest；出现 invalid_reason 时该组不能进入主结论。
- Redis static-vs-Agent 要求 stress-ng 父进程和 worker 全部位于同一 background cgroup；manual_static 必须出现 nr_throttled_delta>0。
- sched_ext 组保存 state、gate_status 和 scx stats；cgroup_v2 组必须确认 sched_ext disabled。
- cpu_per_10k_requests 来自同窗口全系统 /proc/stat，只作为辅助效率指标，不写成目标 cgroup 的独占 CPU 消耗。
- PSI probe 组包含额外负载，只作为门控与调度路径证据，不与无 probe 组直接比较净性能收益。
- 网络、安全和资源测试保存 apply 前、apply 后与 rollback 后状态，并在最终门禁检查无 BPF/LSM/TC/XDP 残留。

4.1.4 Web Console 前后端测试与环境连通性核验

当前封版仓库状态下，Web Console 已完成前端构建、后端测试和 SP4 现场访问验证。前端仍以 React 18、TypeScript、Vite 和 lucide-react 实现，后端以 Node.js 白名单 action、SSE 日志和 evidence 读取为核心；页面可通过 SP4 上的 127.0.0.1:18080 服务和本地 SSH 隧道访问。

本轮已完成 TypeScript 检查、Vite 生产构建、后端安全测试和 SP4 在线页面演示验证，如表 22 所示；Web Console 可展示 Overview、Skills & Agent、Evidence、Policy Engine Timeline、Live Demo 和任务日志。

表 22 Web Console 当前复现结果

测试项	执行结果	实际断言/产物	结论
TypeScript 检查	通过，exit=0	tsc -b --pretty false	类型关系与工程配置可通过编译器检查

测试项	执行结果	实际断言/产物	结论
Vite 生产构建	通过, 1,578 modules	index.html 0.41 kB; CSS 7.39 kB; JS 173.35 kB	六页 React 控制台可生成静态产物
后端测试	10/10 通过	白名单、路径穿越、kind、单任务锁、确认令牌、变更令牌、Content-Type	任务入口与 HTTP 安全边界符合源码断言
页面级在线展示	通过	SP4 控制台启动成功; 通过 SSH 隧道访问; Overview、Skills & Agent、Evidence、Policy Engine Timeline、Live Demo 及任务日志可显示	真实页面可用于答辩演示, 服务仍保持 loopback 安全边界

Web Console 当前以 SP4/123 为主演示环境, 默认监听 127.0.0.1:18080。推荐本地使用 `ssh -N -L 18081:127.0.0.1:18080 openEuler-2403-LTS-SP4` 建立隧道, 再访问 `http://127.0.0.1:18081`。该页面能够触发 `check_env`、`list_skills`、`status_json`、`doctor-safe`、`demo_offline` 和 `policy_engine_lab` 等白名单动作, 并通过 SSE 展示日志; 最终结论仍以 `release manifest` 和 `evidence gate` 为准。

4.2 测试结果

4.2.1 最终质量门禁

最终封版门禁已升级为双环境矩阵和 `release evidence gate`: SP4 主验证环境通过 29/29 P0、100 轮 `smoke` 和 5 轮 `doctor-safe`; SP3 强制兼容环境通过 10/10 `compatibility gate`; `python3 scripts/collect_final_evidence.py --validate-release` 通过, 输出 `entries=42`、`missing_required=0`、`warnings=0`。

最终证据清单、release evidence、SP4/SP3 双环境门禁、formal artifact、release bundle 和仓库同步状态均有数量化结果与证据位置，如表 23 所示。

表 23 最终质量门禁摘要

门禁指标	结果	证据
Release evidence gate	entries=42, missing_required=0, warnings=0	python3 scripts/collect_final_evidence.py - -validate-release
SP4 final gate	29/29 P0 + 100 smoke + 5 doctor-safe	SP4 final gate 日志与 release evidence
SP3 compatibility gate	10/10 通过	SP3 compatibility gate 日志
Formal artifact	tested_code_commit=7a99d87048f4f2040377354 bfe0ce21401664642; artifact_id=ef1bae7ac138acd0eb1a59fc3880ca 550330ef89f87615e8579a0ef264240; RUNS=10 suite	formal build manifest 与 run manifest
Release bundle	已生成并校验；源码包解压后 make agent、 make unit-tests、--validate-config 通过	GitHub Release v0.1-final; release artifact SHA256
仓库同步	GitHub main= 8985340; GitLink master= 4f294b1d; SP4 主仓库为最终验证源	GitHub/GitLink 远端与 SP4 主验证仓库

4.2.2 Redis 混部后端对照

SP4 frozen-code Redis 后端对照使用 10 轮、16 clients、20,000 requests、2 个 stress worker。实验结果显示，cgroup v2 在 Redis GET 和 INCR 混部场景中有清晰收益，PING_INLINE 与 SET 的改善较小，说明 profile 效果与请求类型有关。关键数字统一整理在表 23 中。

同一实验中 noisy_scx_normal、noisy_scx_always_active 和 noisy_scx_psi 在多个测试项上退化；noisy_scx_psi 还包含额外 PSI probe。项目据此把 cgroup v2 作为当前 Redis 混部的主要性能证据，把 sched_ext 结果报告为 valid/regressed 或路径有效性证据，并在最终结论中保留这些负结果。

4.2.3 人工静态与 Agent 动态

独立的 Redis static-vs-Agent 10 轮实验统一把后台父子进程放入 background cgroup。结果显示，Agent 动态控制能够自动进入有效 profile，并在 GET 吞吐上接近或略高于人工静态调参；但 P99 不优于人工静态，PING_INLINE 等指标也不能说明 Agent 永远超过人工最优。因此本实验的结论是“自治治理可达到可用效果，并提供审计与恢复”。

在 Redis GET 混部场景中，cgroup v2、人工静态和 Agent 动态三种控制方式均需同时结合吞吐量与 P99 延迟进行比较，如图 24 所示。

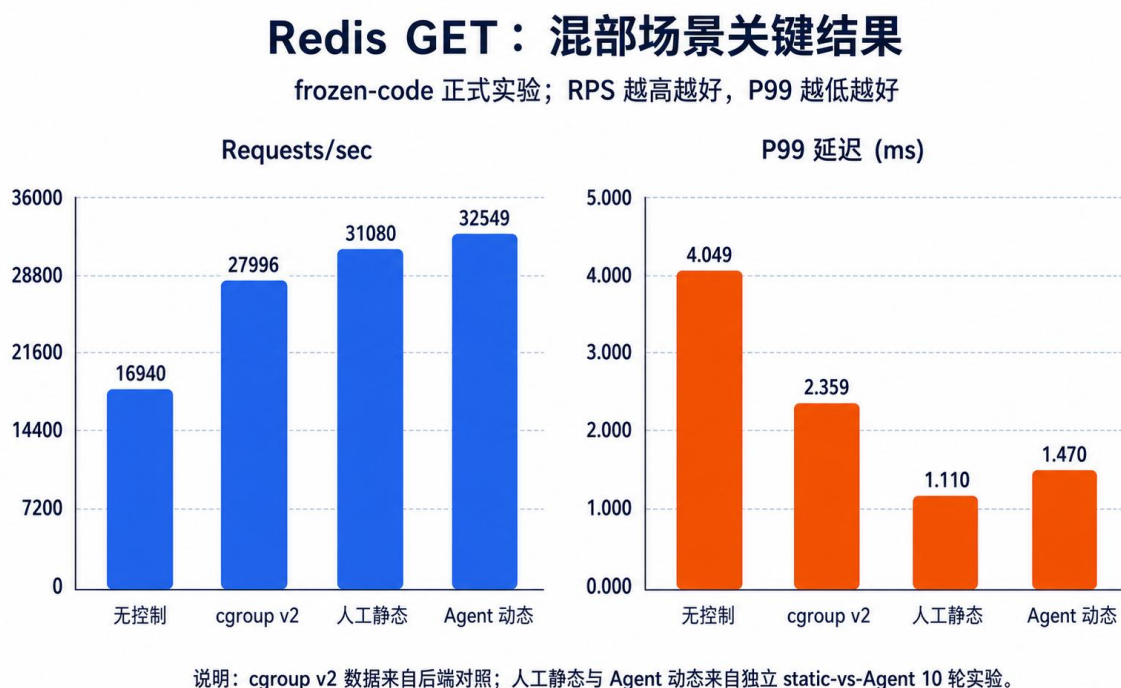


图 24 Redis GET 混部场景关键结果

为便于直接查看，本文将正式性能测试中的核心数字集中整理为表 24。表中保留吞吐、P99、运行参数和边界说明；图表只选择最能解释主结论的场景展开，避免用过多柱状图重复同一组数字。不同实验的来源、参数和结论边界在表中单独说明，避免把不同组别的数据混作同一条结论。

表 24 性能测试关键结果摘要

实验项	关键数据	直接结论	边界说明
Redis 后端 对照: GET	noisy_default 17,817.630 RPS / P99 3.026 ms; cgroup v2 27,995.558 RPS / P99 2.359 ms	RPS +57.12%, P99 降低 22.04%	SP4, RUNS=10, 16 clients, 20,000 requests, 2 stress workers
Redis 后端 对照: INCR	cgroup v2 相对 noisy_default: RPS +69.71%, P99 降低 37.29%	INCR 收益明确	PING_INLINE 与 SET 收益 较小, profile 与请求类型相 关
人工静态 vs Agent: GET	无控制 16,940.094 RPS / P99 4.049 ms; 人工静 态 31,080.280 / 1.210; Agent 32,549.169 / 1.470	Agent 自动控制 达到可用效果, 并保留审计与 rollback	P99 不优于人工静态, 不代 表所有指标最优
压力梯度: 8 workers GET	无控制 9,292.700 RPS / P99 5.439 ms; cgroup v2 33,291.173 / 1.295	压力升高时, 资 源治理收益更明 显	workers=0 时两组接近, 安 静场景不应常驻控制
Mixed- Adaptive	10 轮均观察到 ACTIVE、 COOLDOWN、恢复证 据和 scheduler update; 平均切换延迟 1116.818 ms	同一轮压力触 发、执行和恢复 闭环成立	激活期含 PSI probe, 只作 为门控链路证据

实验项	关键数据	直接结论	边界说明
Nginx 后端 对照	noisy_default 16,644.758 RPS / P99 3.856 ms; cgroup v2 16,200.689 / 3.832	吞吐 -2.67%, P99 仅改善 0.62%	Redis profile 不能直接推广 到 Nginx
throughput- first	default_batch 204,092,569.6 ops/s; cgroup 204,163,072.0; scx 44,130,058.24 observe-only / active	cgroup 近似持 平, scx 明显退 化	throughput_profile 为显式实 验能力, 默认关闭
Agent 自身 开销	cgroup / active scx CPU: 0.400% / 0.487% / 0.537%; RSS 约 7,855 / 7,753 / 7,209 KB	用户态控制面开 销较低	只描述 Agent 进程, 不代表 内核路径无开销

表 25 正式实验结论分层

结论类型	代表结果	可采用的表述	不应采用的表述
可信正向	Redis GET/INCR 在 cgroup v2 后端下吞吐 提升, P99 降低	cgroup v2 动态治理在部分 Redis 混部场景中有效	所有 workload 都稳 定提升
统计中性/ 边界	Nginx cgroup RPS 约 - 2.67%, P99 仅小幅改 善	Nginx 对当前 profile 不敏 感, 需 workload 专用调优	Nginx 也显著提升
明确退化	throughput-first 与多个 scx 组出现明显下降	sched_ext 控制链可运行, 但当前策略仍需调优	scx 已带来稳定性能 优势
功能闭环	Mixed-Adaptive 10 轮 观察到 ACTIVE、 COOLDOWN 与恢复	压力触发、执行和恢复链 路成立	把门控链路结果当 作净性能收益

实验图表采用“少而准”的原则：图 24 保留 Redis GET 的核心正向对比，图 25 展示 Mixed-Adaptive 状态闭环，图 26 展示 Agent 用户态开销；表 24 和表 25 集中承载 Nginx、throughput-first、scx 退化和统计边界。这样处理可以让主要收益、能力边界和负结果同时可见，也避免为每个负向或中性实验单独增加图表导致正文过长。

4.2.4 压力梯度与闭环恢复

Redis pressure-gradient 在 0/1/2/4/8 个 stress worker 档位各运行 3 轮。结果显示，后台压力越高，cgroup v2 抑制干扰的价值越明显；workers=0 时两组接近，说明控制不应在安静场景中无条件常驻。8 worker 档位的关键数字见表 24。

Mixed-Adaptive 10 轮实验覆盖 quiet_pre、pressure_active 和 recovery 三阶段。每轮都观察到 ACTIVE、COOLDOWN、恢复证据和 scheduler update，说明压力触发、策略执行和恢复闭环可以连续运行。激活期包含额外 PSI probe，因此只用于门控链路验证，不作为净性能提升结论。

Mixed-Adaptive 实验按压力前、门控激活和恢复三个阶段展示状态变化，用于验证控制触发与恢复闭环，如图 25 所示。

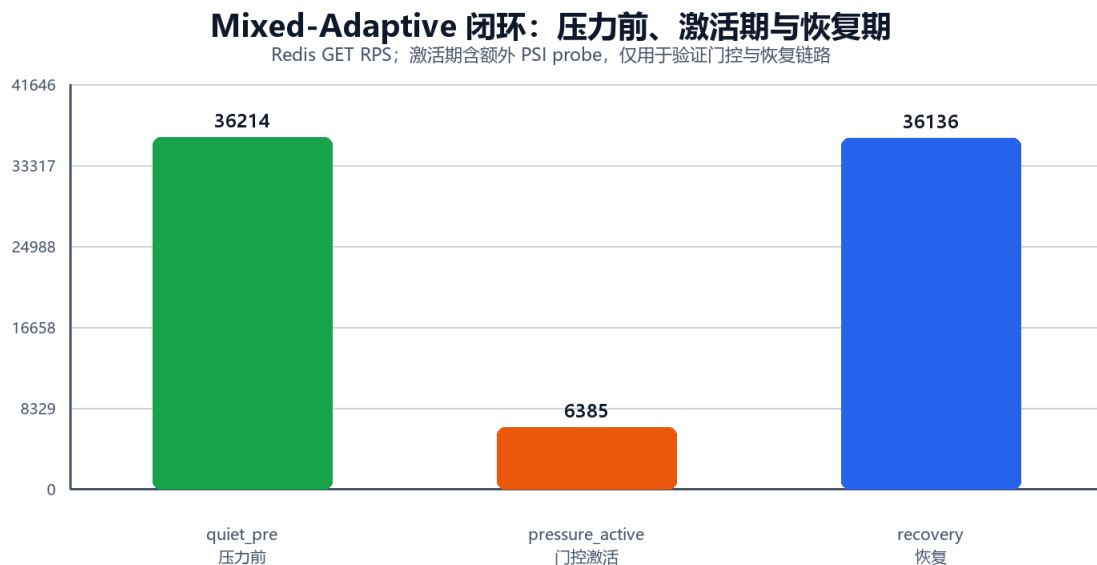


图 25 Mixed-Adaptive 压力前、激活期与恢复期

4.2.5 Nginx 与 throughput-first 边界

Nginx 10 轮结果没有形成稳定正收益：cgroup v2 吞吐略降，P99 变化很小，sched_ext 各组也出现退化。该结果说明 Redis profile 不能直接推广到 Nginx，Nginx 需要单独的分类特征和执行参数。

throughput-first 10 轮轻量 CPU worker 实验中，cgroup 组与默认组近似持平，scx_throughput_first 明显退化。该结果用于说明 throughput_profile 仍属于显式实验能力，封版默认关闭。

4.2.6 Agent 自身开销

Agent overhead 以 /proc/<pid>/stat 的 utime/stime 和 /proc/<pid>/status 的 VmRSS 测量，每组 10 轮、单轮 8 秒。结果显示用户态控制面 CPU 占用低、RSS 规模小，但该结论只描述 Agent 进程本身，不等于内核执行路径没有开销。关键数据见表 23。

observe-only、active cgroup 和 active sched_ext 三种模式的用户态 CPU 开销处于较低水平，但该图不代表内核路径没有开销，如图 26 所示。

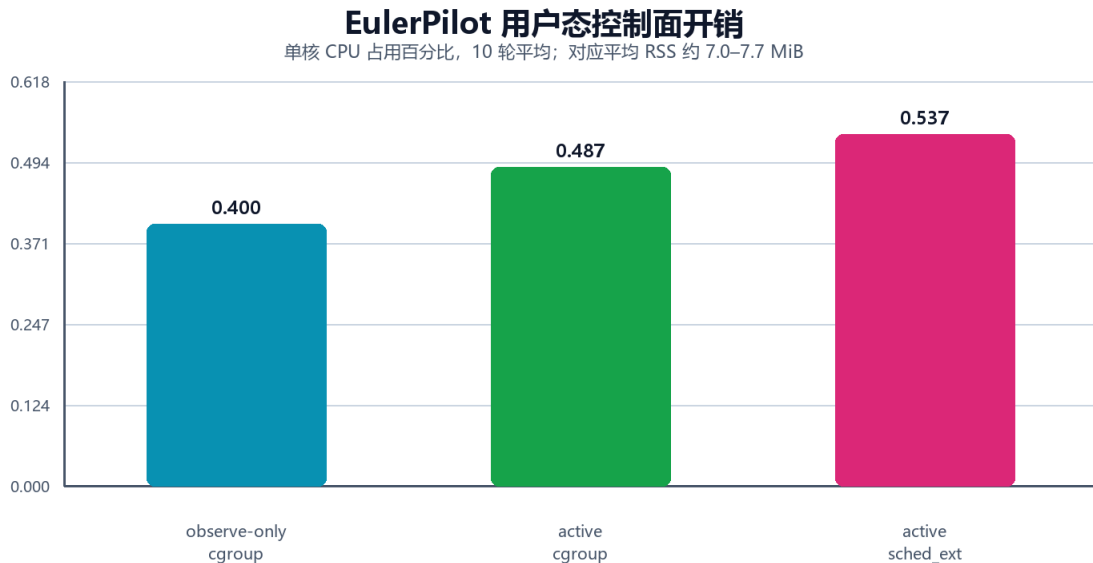


图 26 EulerPilot 用户态控制面 CPU 开销

4.2.7 资源、网络、安全和真实 Pod 验证

Resource、Network、Security、Policy Engine 和真实 Pod 场景均有对应结果目录及 apply—verify—rollback 验证摘要，如表 26 所示。

表 26 跨模块集成验证

能力	代表证据	结果摘要
Resource IO	results/resource_control/io-20260624-*	io.weight/io.max 写入、限速效果与恢复；双机 result=pass
真实 Pod cgroup	real-pod-target-20260630-k3s-*	解析 Pod cgroup，写入资源控制并恢复；双机 pass
真实 Pod host veth QoS/XDP	real-pod-veth-qos/xdp-*	解析 host veth，限速或四规则 drop/pass，cleanup 后恢复；双机 pass
Security anomaly	anomaly-rules/process-filter/combo-scope-*	事件阈值、进程过滤与组合 scope；双机 pass
Credential anomaly	credential-anomaly-*	credential churn 事件聚合；双机 pass
Policy Engine	security-network-resource-*	同一 transaction_id 串联 Security→Resource/Network→rollback；双机 pass
SP4 K8s/Web	results/k8s/sp4-validation-20260708-023552	隔离 namespace、白名单动作、验证后无 EulerPilot 残留

4.2.8 结果限制

- 正式性能测试集中在一台 SP4 主机和 Redis/Nginx/轻量 CPU worker，尚不能代表所有硬件、内核和业务负载。
- 部分实验报告保留“后续扩大 RUNS”的历史模板语句，但 2026-07-24 目录实际为 RUNS=10；本文以目录、manifest 和 CSV 为准。

- sched_ext 当前主要证明集成路径和调度状态，现有策略在 Redis/Nginx/批处理多个场景退化，不能作为通用性能优势。
- PSI probe 为稳定触发门控引入额外负载，相关数据只用于状态机和恢复证据。
- Deep credential hooks 已完成 scoped attach，但普通用户态 workload 未稳定产生 runtime hit；证据明确记录 no-userland-hit-observed。

第 5 章 实现难点说明

5.1 跨内核能力差异与优雅降级

难点在于 openEuler 发行环境、OLK 对照内核和 SP4 自编译 sched_ext 内核具备不同能力。sched_ext 如果作为启动必需项，作品会在发行默认内核上失败；如果完全忽略 sched_ext，又无法验证可编程调度路径。项目通过 CapabilityDetector、Skill::probe 和 backend fallback 把能力差异显式化：cgroup v2 是稳定主路径，scx 只有在二进制可执行、/sys/kernel/sched_ext 与所需 BPF 能力存在时启用；doctor-safe、status 和最终门禁均会记录降级原因。这样做的实际意义是，SP3 发行内核仍能完成资源治理和安全 smoke，SP4 自编译内核则用于验证 scx 增强路径，二者不是相互替代关系。

5.2 任务身份与 PID 复用

BPF map 以 PID 为索引时，短生命周期进程会导致同一数字 PID 被复用。如果直接对累计 runtime/wait 做差，新进程可能继承旧进程基线并产生巨大错误增量。实现以 start_boottime_ns 作为身份源，缺失时使用 generation cookie，并在计数器回退或身份变化时安全重置。用户态还需要先按 TID 计算 delta，再按 TGID 聚合，否则多线程进程会出现线程退出、重建后样本串联的问题。单元测试覆盖正常累计、计数回退、PID 复用和 fallback generation 变化。

5.3 TargetResolver 的容器与 Pod 映射

容器与 Kubernetes 的用户可见名称并非内核控制器直接接受的对象。Resource Control 需要 cgroup 路径，Network Policy 需要宿主机 veth。实现把解析分成运行时查询、PID/cgroup 定位、netns/iflink 反查三个阶段，并为 docker、podman、isula、containerd/crictl 与 kubectrl 提供不同分支。解析成功后仍要核对 namespace、Pod UID、container 名称、container ID、ifindex 与 cgroup 路径边界；解析失败时保持 blocked/unavailable，系统不会自动安装运行时、拉取镜像或操作生产 namespace。

5.4 多资源事务一致性

跨 Skill 动作的难点在于半成功状态的恢复。例如资源降级成功后 TC attach 失败，如果没有事务日志，目标会长期处于半配置状态。Policy Engine 为一次触发生成

transaction_id，每项动作在执行前保存旧值或句柄，执行后回读；失败、rollback 或 stop 时逆序恢复。这里的事务定位为面向内核对象的 best-effort transaction：能恢复的动作必须保存快照和校验结果，不可逆动作如 memory.reclaim 默认关闭并单独确认。ActionJournal 把每项动作的 APPLIED/ROLLED_BACK 状态落盘，使恢复结果可审计。

5.5 eBPF Hook 生命周期与无残留

TC、XDP、LSM 和 sched_ext 的 attach 对象分布在不同内核子系统，清理方式也不同。项目不把 LSM link 持久 pin 到 bpffs；Network Skill 保存 qdisc/filter、XDP attach 和 veth 句柄；scx 用户态进程由 ScxExecutor 记录启动身份和退出状态。清理阶段不能简单执行 pkill 或删除所有 qdisc，而要判断对象是否由本事务创建、当前 ifindex 或 program id 是否匹配。每个集成测试都保存 before/after/rollback 状态，最终门禁再次检查 BPF/LSM/TC/XDP 残留，形成“功能通过并且现场恢复”的双条件。

5.6 实验有效性与负结果报告

系统实验容易受到 CPU 频率、缓存、运行顺序、后台任务归属和探针本身的影响。项目通过平衡轮换、多轮平均与标准差、controlled_pids/cgroups、throttle 计数、sched_ext state、invalid_reason 和 frozen-code commit 限制偏差。更重要的是，实验有效性和性能好坏分开判定：benchmark 正常完成但延迟很高，属于真实负结果；调度计数缺失、工件哈希不一致或 cleanup 失败，才判为 invalid。报告同时保留 Nginx、scx 和 throughput-first 的退化结果，并把额外 PSI probe 组从净性能比较中分离，避免只挑选有利指标。

5.7 控制台的最小权限交互

控制台界面如果直接接受任意 shell 命令，会扩大系统级 Agent 的攻击面。后端因此不提供自由命令接口，只加载 actions.yaml 白名单；可变更动作要求 token 和与 action/参数/会话绑定的一次性确认令牌；请求体限制 4096 字节，检查 JSON Content-Type、Origin 和 Fetch Metadata；JobManager 限制单任务、运行时间和输出大小。现场演示时，页面上的按钮本质上是对已经审核脚本的受控入口，失败、超时或取消后仍要进入 cleanup 和 job 状态记录。该设计定位为实验室旁路控制台，不宣称已达到互联网多租户控制面的安全等级。

第 6 章 总 结

6.1 完成度

EulerPilot 已完成从内核观测、用户态分析、双后端执行、Skills 生命周期、TargetResolver、跨 Skill 事务、审计回滚到正式实验和证据展示的工程闭环。最终证据校验结果为 entries=42、missing_required=0、warnings=0；SP4 主验证环境通过 29/29 项 P0 门禁、100 轮 smoke 和 5 轮 doctor-safe，SP3 强制兼容环境通过 10/10 项兼容性门禁。Release bundle 已完成生成、校验和独立解压验证，构建、单元测试及配置校验均通过。综合来看，本项目已经具备源码可构建、功能可运行、结果可复现、风险可回滚、材料可提交的封版交付条件。

6.2 主要结论

- 在本项目 Redis 混部场景中，cgroup v2 控制明显改善 GET/INCR 吞吐和 P99，压力梯度越高收益越明显。
- Agent 动态控制能够自动进入有效 profile、写入、审计并 rollback；用户态控制面 CPU 平均低于单核 0.6%，RSS 约 7–8 MiB。
- Mixed-Adaptive 10 轮均观察到 ACTIVE、COOLDOWN、scheduler update 与恢复，平均切换延迟约 1.117 s。
- Nginx、throughput-first 与多个 sched_ext 组出现退化，证明当前策略需要 workload 特定调优；项目没有把路径可用性等同于性能优势。
- Resource、Network、Security 和 Policy Engine 已在显式 cgroup、真实 Pod cgroup 与 host veth 上形成 apply—verify—rollback 证据。

6.3 创新点归纳

本作品的创新点集中在以下七个方面，每一点都对应到具体实现和验证材料。

1. 资源管控 Agent 闭环。系统把内核观测、用户态判断、Skill 执行、写后验证和失败恢复连成一条链，面向混部环境中的卡顿、抢资源和难排查问题给出可运行方案。

2. 可解释的自适应评分。Analyzer 不依赖离线训练模型，直接使用等待比例、唤醒次数、运行时长、迁移次数和 PSI 等实时指标计算 latency_score、batch_score、interference_score，再交给 PsiGate 做滞回控制，便于现场解释每一次决策。
3. 统一 Skill Framework。Resource、Scheduling、Network、Security 共用 configure、probe、start、snapshot、rollback、stop 等生命周期，入口、状态和清理流程保持一致，新增 Skill 不需要改写 Agent 主循环。
4. 稳定主路径与增强路径协同。cgroup v2 承担发行内核上的稳定资源治理，sched_ext/scx 在满足能力条件时提供可编程调度验证，并通过 graceful fallback 保证不可用时系统仍可运行。
5. 面向真实目标的解析与作用域控制。TargetResolver 将 PID、cgroup、容器和 Kubernetes Pod 统一为可执行目标，同时对 host veth、外部 cgroup、TC/XDP 和安全规则设置明确边界。
6. 跨 Skill 事务和可审计恢复。Policy Engine、AuditBus 与 ActionJournal 记录 request_id、transaction_id、动作旧值、新值和恢复状态，使一次资源、网络或安全联动可以从触发事件追踪到最终回滚。
7. 证据优先的交付方法。项目保留正向收益、统计中性和退化结果，使用 tested commit、formal artifact、SP4/SP3 双环境门禁和 entries=42 的 evidence 校验保证结论可复查。

6.4 后续工作

- 针对 Nginx 和批处理建立独立的特征与 profile，不复用 Redis 参数；后续使用更多 workload 组合做阈值校准和回归验证，优先保持规则可解释性。
- 重构 scx_eulerpilot 的 DSQ 分流和 dispatch 逻辑，先修复 enqueue/dispatch 计数与吞吐退化，再扩大 sched_ext 正向结论。
- 在不同 CPU 架构、核心数和内存/IO 设备上重复 frozen-code 实验，报告置信区间、效应量和跨主机稳定性。
- 把 TargetResolver 与策略 schema 的模糊测试、路径遍历测试和容器运行时兼容矩阵加入持续集成。

- 将当前 JSONL/CSV 证据 schema 版本化，自动生成文档中的表格和图，减少源码、实验和报告之间的口径漂移。

6.5 致谢与开源边界

本项目感谢 openEuler、Linux Kernel、libbpf、Kubernetes、XDP、iproute2 等开源社区长期积累的系统接口、文档和示例，也感谢比赛平台提供的开放题目、评审反馈和复现要求。项目开发和验证过程中，亦获得了来自测试环境维护、脚本复现、文档审阅和演示反馈等方面的帮助。

项目使用 Linux 内核提供的 eBPF、PSI、cgroup v2、sched_ext、TC/XDP 与 BPF LSM 接口，并使用 libbpf、yaml-cpp、React、Vite 等开源组件。third_party/reference 中的 libbpf-bootstrap、LMP、Kata-LSM-eBPF 和 PerfInsight PSI 快照仅用于设计对照，保留各自许可证与来源说明，不计入 EulerPilot 原创生产代码。项目默认使用 MulanPSL-2.0 发布，文件自带 SPDX 标识时以文件声明为准。

参 考

- [1] Humphries, J. T., Natu, N., Chaugule, A., Weisse, O., Rhoden, B., Don, J., Rizzo, L., Rombakh, O., Turner, P. and Kozyrakis, C. ghOSt: Fast & Flexible User-Space Delegation of Linux Scheduling. Proc. 28th ACM Symposium on Operating Systems Principles (SOSP), 588-604, 2021. DOI: 10.1145/3477132.3483542.
- [2] Linux Kernel. PSI - Pressure Stall Information. Linux Kernel Documentation. <https://docs.kernel.org/accounting/psi.html>.
- [3] Linux Kernel. Control Group v2. Linux Kernel Documentation. <https://docs.kernel.org/admin-guide/cgroup-v2.html>.
- [4] Linux Kernel. Extensible Scheduler Class. Linux Kernel Documentation. <https://docs.kernel.org/scheduler/sched-ext.html>.
- [5] openEuler. openEuler 24.03 LTS SP3 Server Documentation and Release Notes. https://docs.openeuler.org/en/docs/24.03_LTS_SP3/server/index.html.
- [6] openEuler. openEuler 24.03 LTS SP4 Server Documentation and Release Notes. https://docs.openeuler.org/en/docs/24.03_LTS_SP4/server/index.html.
- [7] Linux Kernel. LSM BPF Programs. Linux Kernel Documentation. https://docs.kernel.org/bpf/prog_lsm.html.
- [8] Linux Kernel. BPF Documentation. Linux Kernel Documentation. <https://docs.kernel.org/bpf/>.
- [9] libbpf. libbpf: BPF CO-RE and userspace library. docs.ebpf.io. <https://docs.ebpf.io/ebpf-library/libbpf/>.
- [10] libbpf-bootstrap. Scaffolding for BPF application development with libbpf and BPF CO-RE. GitHub. <https://github.com/libbpf/libbpf-bootstrap>.
- [11] Kubernetes. Pods. Kubernetes Documentation. <https://kubernetes.io/docs/concepts/workloads/pods/>.
- [12] iproute2. tc(8) - show / manipulate traffic control settings. man-pages. <https://man7.org/linux/man-pages/man8/tc.8.html>.
- [13] The XDP Project. XDP Tutorial. GitHub. <https://github.com/xdp-project/xdp-tutorial>.
- [14] openEuler Community. Mulan Permissive Software License v2. <https://license.coscl.org.cn/MulanPSL2/>.